UNIVERSIDADE PAULISTA – UNIP
CURSO DE CIÊNCIA DA COMPUTAÇÃO

ALUISIO PEREIRA ALVES N135891
ENZO ORLANDI GOMES G788EJ5
ERICK DE BRITO CARVALHO G78HED3
KAYKY CRESPO DOS SANTOS G839226

CINTO INTELIGENTE COM IoT E INTELIGÊNCIA ARTIFICIAL PARA MONITORAMENTO DE ATIVIDADES, SINAIS VITAIS E LOCALIZAÇÃO INDOOR

Sistema de Detecção de Possíveis Quedas

Guia do Usuário e Manual de Operação

SÃO PAULO
2026

ALUISIO PEREIRA ALVES – RA N135891
ENZO ORLANDI GOMES – RA G788EJ5
ERICK DE BRITO CARVALHO – RA G78HED3
KAYKY CRESPO DOS SANTOS – RA G839226

CINTO INTELIGENTE COM IoT E INTELIGÊNCIA ARTIFICIAL PARA MONITORAMENTO DE ATIVIDADES, SINAIS VITAIS E LOCALIZAÇÃO INDOOR

Sistema de Detecção de Possíveis Quedas

Guia do Usuário e Manual de Operação do conjunto prototipado, apresentado ao Curso de Ciência da Computação da Universidade Paulista – UNIP como entregável do Trabalho de Conclusão de Curso, na vertente SmartCity – Cidades Inteligentes e Infraestrutura Urbana.

Orientador: Prof. Marco Gomes

SÃO PAULO

2026

SUMÁRIO

Seção 1 — Identificação do Ecossistema e Vertente 4
1.1 Identificação 4
1.2 Enquadramento na vertente SmartCity 4
1.3 Diagrama de arquitetura de alto nível 7
1.4 Fluxo de funcionamento 8
Seção 2 — Manual de Instalação e Montagem do Hardware 9
2.1 Lista de materiais (BOM) 9
2.2 Pinagem (pinout) 10
2.3 Diagrama esquemático e montagem 12
2.4 Especificações de alimentação 18
2.5 Ordem de montagem 25
2.6 Fixação mecânica e encapsulamento 28
Seção 3 — Guia de Instalação e Configuração do Software 29
3.1 Requisitos mínimos 29
3.2 Configuração da borda (firmware) 31
3.2.1 Instalar a Arduino IDE e o pacote ESP32 31
3.2.2 Drivers USB e reconhecimento da porta COM 31
3.2.3 Configurar e gravar o firmware da antena (`Arduino/anchor_mvp_v1/anchor_mvp_v1.ino`) 32
3.2.4 Configurar e gravar o firmware do cinto (`Arduino/cinto_mvp_v1/cinto_mvp_v1.ino`) 33
3.3 Subida do servidor, backend e banco de dados 34
3.3.1 Broker Mosquitto 34
3.3.2 Backend Python 34
3.3.3 Banco de dados 35
3.3.4 Variáveis de ambiente 35
3.4 Aplicação cliente (painel) 38
Seção 4 — Módulo de Inteligência Artificial 39
4.1 Objetivo operacional 39
4.2 Dataset de treinamento e validação 39
4.3 Arquitetura do modelo e treinamento 40
4.4 Resultados e métricas 41
4.5 Declaração formal de uso de IA assistiva 43
Seção 5 — Manual de Operação e Diagnóstico 45

5.1 Partida e calibração 45
5.2 Operação diária 47
5.2.1 Procedimento ao receber um alerta 56
5.3 Resolução de problemas 56
5.4 Desligamento seguro e manutenção preventiva 58
5.5 Pendências de validação antes da entrega 59
5.6 Registros audiovisuais e rastreabilidade dos testes 60
Referências 62

SEÇÃO 1 — IDENTIFICAÇÃO DO ECOSSISTEMA E VERTENTE

1.1 Identificação

A solução apresentada neste guia é o Cinto Inteligente com IoT e Inteligência Artificial para Monitoramento de Atividades, Sinais Vitais e Localização Indoor, um sistema de detecção de possíveis quedas desenvolvido como Trabalho de Conclusão de Curso do Bacharelado em Ciência da Computação da Universidade Paulista (UNIP), em 2026, sob orientação do Prof. Marco Gomes.

O projeto está enquadrado na vertente SmartCity – Cidades Inteligentes e Infraestrutura Urbana.

Este manual descreve a implementação disponível no repositório: aquisição inercial a 100 Hz, faixas de ±8 g e ±2000 °/s, transmissão dos sensores por GATT, medição de RSSI por GATT/Advertising e armazenamento em SQLite. O ECG é apresentado como sinal bruto experimental; não há cálculo de frequência cardíaca neste firmware. A parte escrita de apoio ainda contém descrições de uma arquitetura anterior (MySQL, sensores por Advertising, 4 Hz, ±2 g e cálculo de BPM), cuja atualização precisa ser confirmada pela equipe. Essas descrições não devem substituir as configurações desta versão.

A equipe executora é composta pelos seguintes integrantes, com as respectivas funções no projeto:

- Aluisio Pereira Alves (RA N135891) – Pesquisador e redator;
- Enzo Orlandi Gomes (RA G788EJ5) – Responsável pelo sistema embarcado;
- Erick de Brito Carvalho (RA G78HED3) – Arquiteto de soluções e desenvolvedor de software;
- Kayky Crespo dos Santos (RA G839226) – Product owner, gerente de projeto e Scrum master.

1.2 Enquadramento na vertente SmartCity

O envelhecimento da população transforma o cuidado com idosos em um serviço urbano: instituições de longa permanência, unidades de saúde e programas

de atendimento domiciliar dependem de equipes que não conseguem acompanhar todas as pessoas o tempo todo. Na classificação de Giffinger et al. (2007), esse domínio corresponde à dimensão de qualidade de vida (*smart living*) da cidade inteligente, e a saúde conectada é um dos principais serviços urbanos apoiados por IoT (Zanella et al., 2014; Gubbi et al., 2013).

O sistema é uma infraestrutura de sensoriamento para esses espaços, com as mesmas características das redes de telemetria urbana:

- Rede fixa de sensores: antenas instaladas no prédio captam o sinal do cinto e medem a intensidade do sinal recebido (RSSI) em cada ponto do ambiente.
- Transporte padronizado: os dados trafegam por Wi-Fi e MQTT, protocolo de publicação e assinatura usado em telemetria de cidades inteligentes.
- Processamento local: a classificação e a estimativa de localização são executadas no computador do ambiente, sem depender de processamento em nuvem. A notificação pelo Telegram utiliza um serviço externo e requer conexão com a internet. A ausência de câmeras não comprova, por si só, conformidade com a LGPD; o tratamento dos dados de saúde exige avaliação específica de acesso, finalidade, retenção e compartilhamento.
- Serviço de alerta: o sistema solicita o envio de uma notificação de possível queda, acompanhada da região estimada quando disponível. O tempo de entrega ponta a ponta não foi quantificado neste manual e depende da comunicação e do serviço externo.
- Escalabilidade: a mesma rede pode ser replicada em vários prédios e, como evolução futura, integrada a centrais de teleassistência do município.

Problema de pesquisa: a demora em perceber uma possível queda de pessoa idosa que vive ou permanece sozinha em ambiente interno. Investiga-se uma alternativa de alerta automático, em contraste com o acionamento manual, combinada com estimativa de localização interna. O protótipo permanece sujeito a falsos positivos, falsos negativos e indisponibilidade de comunicação.

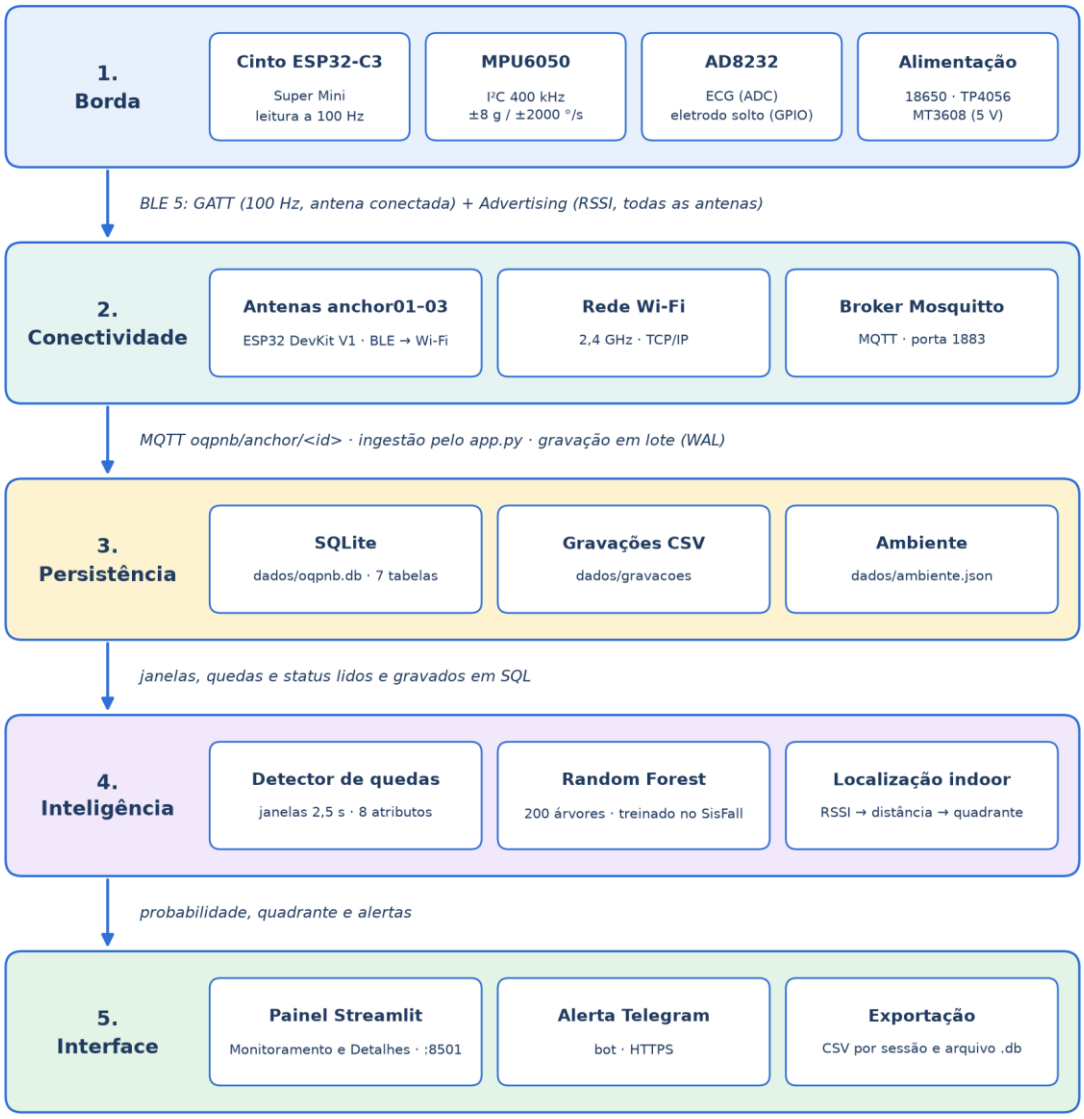
Diferenciais da proposta: alerta automático, que não depende de quem caiu; privacidade, sem câmeras; e inovação frugal, com componentes de baixo custo e fáceis de encontrar (ESP32, MPU6050, AD8232), acessíveis a instituições e famílias.

Hipótese orientadora: um cinto com sensor inercial na cintura, classificado por Random Forest treinado em dataset público, detecta possíveis quedas com alta sensibilidade e, combinado às antenas BLE, informa a região aproximada do ambiente onde a queda ocorreu.

A sensibilidade foi avaliada por ensaio no SisFall (Seção 4.4), não em uso clínico contínuo. A hipótese de localização exige comparação com posições de referência e não é confirmada apenas pelos ensaios de RSSI. A integração funcional, a precisão da localização e a latência devem ser avaliadas separadamente; não se considera a hipótese integralmente comprovada pelas métricas do classificador.

1.3 Diagrama de arquitetura de alto nível

Figura 1 – Arquitetura do ecossistema em camadas



Fonte das figuras: elaborado pelos autores (2026).

Quadro 1 – Camadas, componentes e protocolos do ecossistema

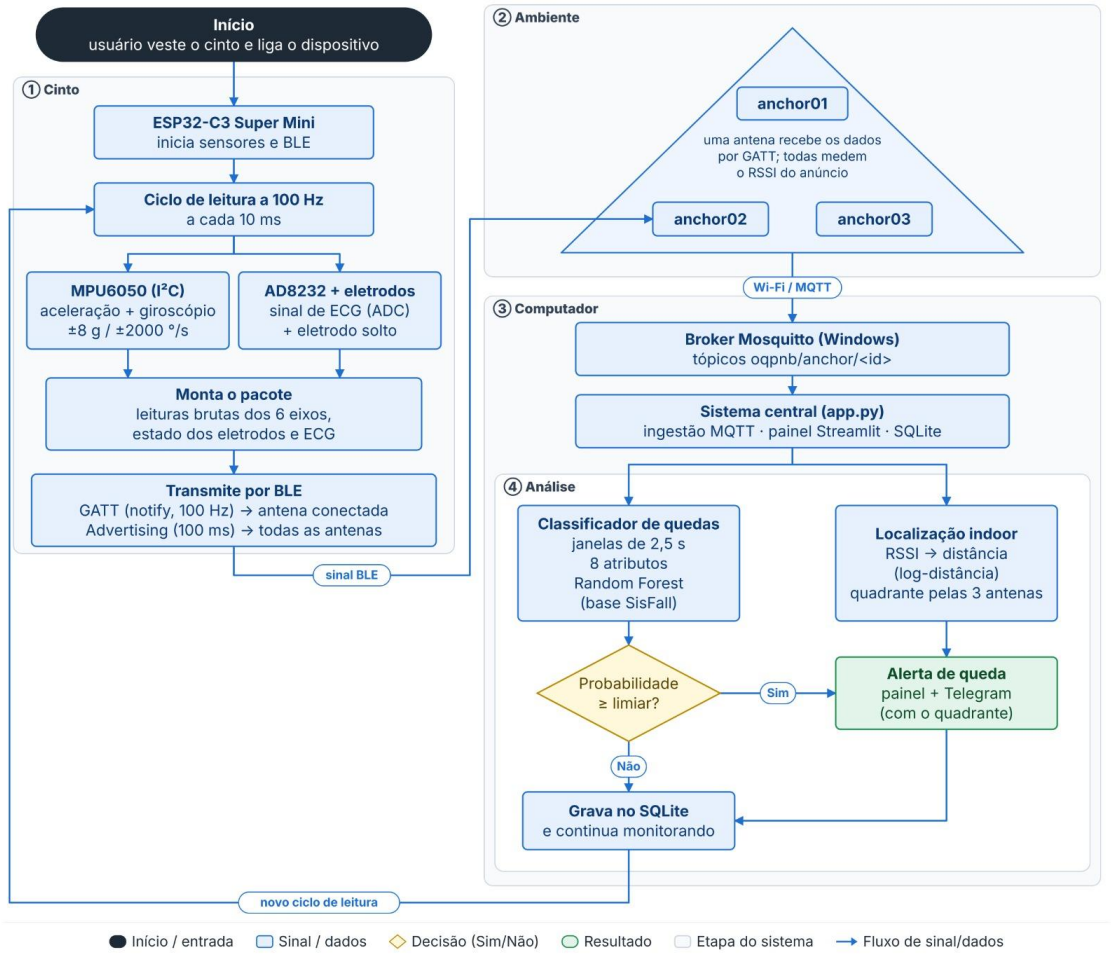
Camada	Componentes	Protocolos
Borda	Cinto (ESP32-C3, MPU6050, AD8232)	I²C (MPU), ADC e GPIO (AD8232)
Conectividade	3 antenas ESP32, roteador Wi-Fi, Mosquitto	BLE 5 (GATT + Advertising), Wi-Fi 802.11 b/g/n 2,4 GHz, TCP/IP, MQTT 3.1.1
Inteligência	`detector_queda.py`, `features.py`, `localizacao.py`, modelo `modelos/modelo_mvp_8g.joblib`	—
Persistência	SQLite (`banco.py`), CSV	SQL
Interface	Painel Streamlit (`app.py`), bot do Telegram (`alerta_telegram.py`)	HTTP (navegador), HTTPS (API do Telegram)

Fonte: Elaborado pelos autores (2026).

1.4 Fluxo de funcionamento

A Figura 2 mostra o caminho de um ciclo de leitura, do sensor no cinto até o alerta.

Figura 2 – Fluxograma do funcionamento completo do sistema



Fonte das figuras: elaborado pelos autores (2026).

SEÇÃO 2 — MANUAL DE INSTALAÇÃO E MONTAGEM DO HARDWARE

2.1 Lista de materiais (BOM)

Cinto (1 unidade)

Quadro 2 – Lista de materiais do cinto

Qtd	Componente	Modelo / part number	Tensão de operação	Função no circuito
1	Microcontrolador	ESP32-C3 Super Mini (Espressif ESP32-C3, RISC-V, Wi-Fi + BLE 5)	5 V no pino 5V (regulador interno para 3,3 V)	Lê os sensores a 100 Hz e transmite por BLE
1	Sensor inercial	MPU6050 (módulo GY-521)	3,3 V	Aceleração e velocidade angular (±8 g / ±2000 °/s)
1	Front-end de ECG	AD8232 (módulo SparkFun ou equivalente) + cabo P2 (3,5 mm) de 3 vias	3,3 V	Sinal de ECG e detecção de eletrodo solto
3	Eletrodos	Eletrodo descartável de ECG com gel (Ag/AgCl), encaixe de pressão	—	Contato com a pele (RA, LA, RL)
1	Carregador de bateria	TP4056 com proteção (DW01A), entrada USB-C	5 V de entrada; 4,2 V de corte	Carrega as baterias e as protege
1	Conversor elevador	MT3608 (step-up)	Entrada 2–24 V; saída ajustada para 5,0 V	Eleva 3,7 V das baterias para 5 V
2	Bateria	Li-ion 18650, 3,7 V, 1000 mAh cada, em paralelo (2000 mAh no total)	3,0–4,2 V	Alimentação portátil
1	Chave	Mini chave gangorra (KCD11 ou equivalente)	—	Liga/desliga
1	LED + resistor (opcional)	LED 3 mm + resistor 330 Ω	3,3 V	Indica eletrodo solto (GPIO 10 no firmware; não aparece na montagem atual da Figura 3)
1	Caixa	Case impresso em 3D (base + tampa)	—	Proteção e fixação no cinto
2–4	Insertos e parafusos	Rosca de latão M3 + parafuso M3	—	Fixação da tampa
—	Fios	Fio de cobre flexível (vermelho/preto para energia; cores diversas para sinal)	—	Ligações soldadas

Fonte: Elaborado pelos autores (2026).

Antena (por unidade; montar 3: Anchor 1, Anchor 2, Anchor 3)

Quadro 3 – Lista de materiais por antena

Qtd	Componente	Modelo / part number	Tensão de operação	Função no circuito
1	Microcontrolador	ESP32 DevKit V1 (ESP32-WROOM-32, conversor USB-serial CP2102, Micro-USB)	5 V no pino VIN	Recebe BLE do cinto, mede RSSI e publica por Wi-Fi/MQTT
1	Carregador de bateria	TP4056 com proteção, USB-C	5 V de entrada	Carrega a bateria
1	Conversor elevador	MT3608	Saída ajustada para 5,0 V	Eleva 3,7 V para 5 V
1	Bateria	Li-ion 18650, 3,7 V, 1000 mAh	3,0–4,2 V	Alimentação portátil
1	Chave	Mini chave gangorra	—	Liga/desliga
1	Caixa	Case impresso em 3D com tampa perfurada e identificação gravada	—	Proteção e fixação
2	Insertos e parafusos	Rosca de latão M3 + parafuso M3	—	Fixação da tampa
—	Fios	Fio de cobre	—	Ligações soldadas

Fonte: Elaborado pelos autores (2026).

Infraestrutura

Quadro 4 – Infraestrutura e acessórios necessários

Qtd	Item	Observação
1	Computador central	Windows 10/11 64 bits; executa broker, backend e painel
1	Roteador ou ponto de acesso Wi-Fi	Rede de 2,4 GHz (o ESP32 não opera em 5 GHz)
1	Cabo USB-C (dados)	Gravação do cinto
1	Cabo Micro-USB (dados)	Gravação das antenas

Fonte: Elaborado pelos autores (2026).

2.2 Pinagem (pinout)

Cinto — ESP32-C3 Super Mini ↔ periféricos (definida em `Arduino/cinto_mvp_v1/cinto_mvp_v1.ino`)

Quadro 5 – Pinagem do ESP32-C3 e dos periféricos do cinto

Pino do ESP32-C3	Pino do periférico	Sinal	Observação
3V3	MPU6050 VCC	Alimentação 3,3 V	
GND	MPU6050 GND	Terra	
GPIO 8	MPU6050 SDA	Dados I²C	I²C padrão do ESP32-C3 (`Wire.begin()` sem argumentos), 400 kHz
GPIO 9	MPU6050 SCL	Clock I²C	Endereço do MPU: `0x68` (AD0 em GND)
3V3	AD8232 3.3V	Alimentação 3,3 V	
GND	AD8232 GND	Terra	
GPIO 4 (ADC)	AD8232 OUTPUT	Sinal de ECG analógico	ADC de 12 bits (0–4095)
GPIO 2	AD8232 LO−	Eletrodo solto (−)	Entrada digital
GPIO 3	AD8232 LO+	Eletrodo solto (+)	Entrada digital
GPIO 10	LED (ânodo, via resistor de 330 Ω)	Indicador	Aceso = eletrodo solto (opcional)
5V	MT3608 VOUT+	Alimentação 5 V	
GND	MT3608 VOUT−	Terra comum	

Fonte: Elaborado pelos autores (2026).

Cinto — alimentação

Quadro 6 – Ligações de alimentação do cinto

Origem	Destino
Baterias 18650 (+), em paralelo	TP4056 B+
Baterias 18650 (−)	TP4056 B−
TP4056 OUT+	Chave gangorra (terminal 1)
Chave gangorra (terminal 2)	MT3608 VIN+
TP4056 OUT−	MT3608 VIN−
MT3608 VOUT+ (5,0 V)	ESP32-C3 5V
MT3608 VOUT−	ESP32-C3 GND
Cabo P2 dos eletrodos	Conector de 3,5 mm do AD8232 (RA, LA, RL)

Fonte: Elaborado pelos autores (2026).

Antena — ESP32 DevKit V1

A antena não possui sensores periféricos; suas ligações externas correspondem ao circuito de alimentação.

Quadro 7 – Ligações de alimentação da antena

Origem	Destino
Bateria 18650 (+) / (−)	TP4056 B+ / B−
TP4056 OUT+	Chave gangorra → MT3608 VIN+
TP4056 OUT−	MT3608 VIN−
MT3608 VOUT+ (5,0 V)	ESP32 VIN
MT3608 VOUT−	ESP32 GND

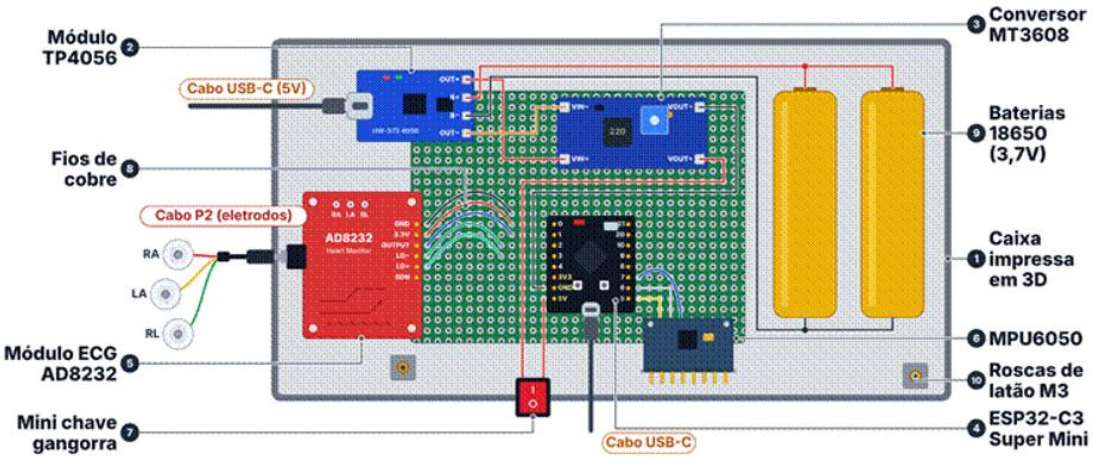
Fonte: Elaborado pelos autores (2026).

2.3 Diagrama esquemático e montagem

Os esquemas elétricos mostram as conexões de alimentação e de sinal dos dois dispositivos. Em ambos, a bateria recarregável é ligada ao módulo TP4056, responsável pela carga, e ao conversor MT3608, que fornece 5 V ao circuito; a chave gangorra permite ligar e desligar o conjunto. No cinto, o ESP32-C3 recebe os dados de movimento do MPU6050 pela comunicação I²C e o sinal cardíaco do AD8232, que também indica quando os eletrodos estão soltos. Na antena, o ESP32 DevKit V1 funciona como unidade de comunicação e não possui sensores conectados. Consulte as tabelas de ligações desta seção para conferir os pinos e sinais.

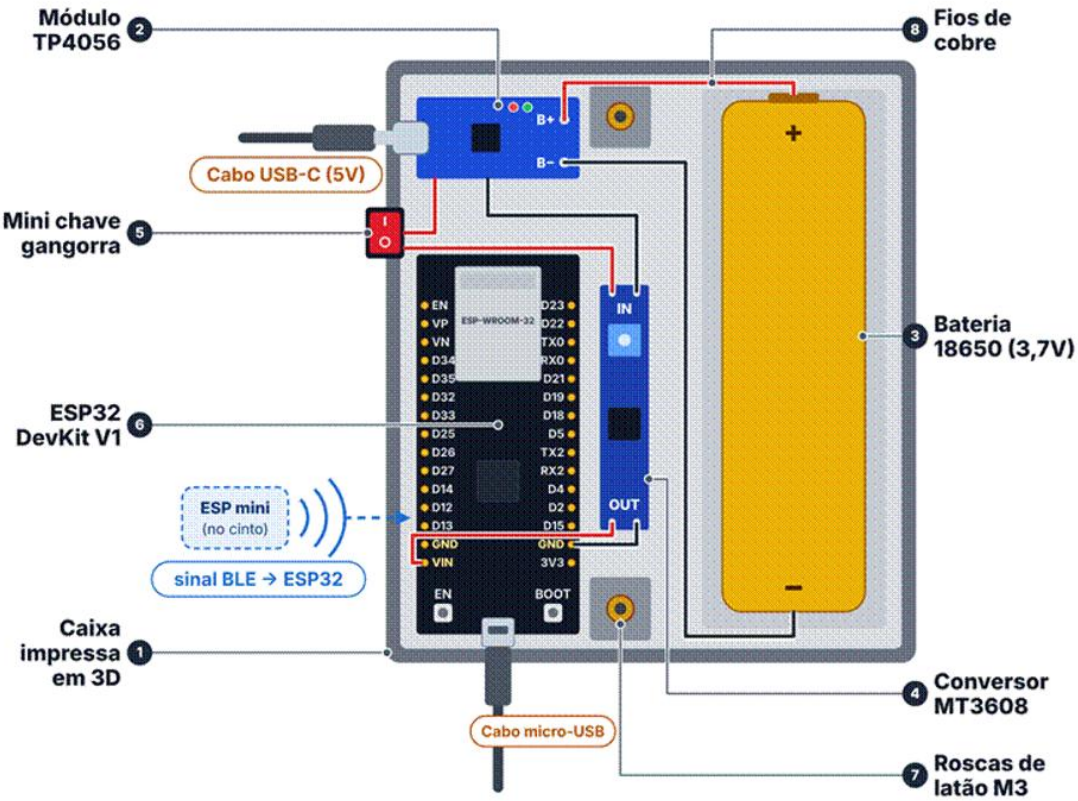
> Pendência de validação elétrica: nas Figuras 5 e 6, a chave gangorra aparece após o MT3608; nas tabelas da Seção 2.2 e nos passos da Seção 2.5, aparece antes do conversor. A equipe deve confirmar a ligação física e alinhar desenhos e instruções antes de permitir sua reprodução por terceiros. Se a chave estiver após o MT3608, o conversor poderá permanecer alimentado com a chave desligada. Não usar as representações divergentes como instrução final de montagem nem presumir isolamento entre USB e bateria sem verificar o circuito real.

Figura 3 – Disposição dos módulos no case do cinto



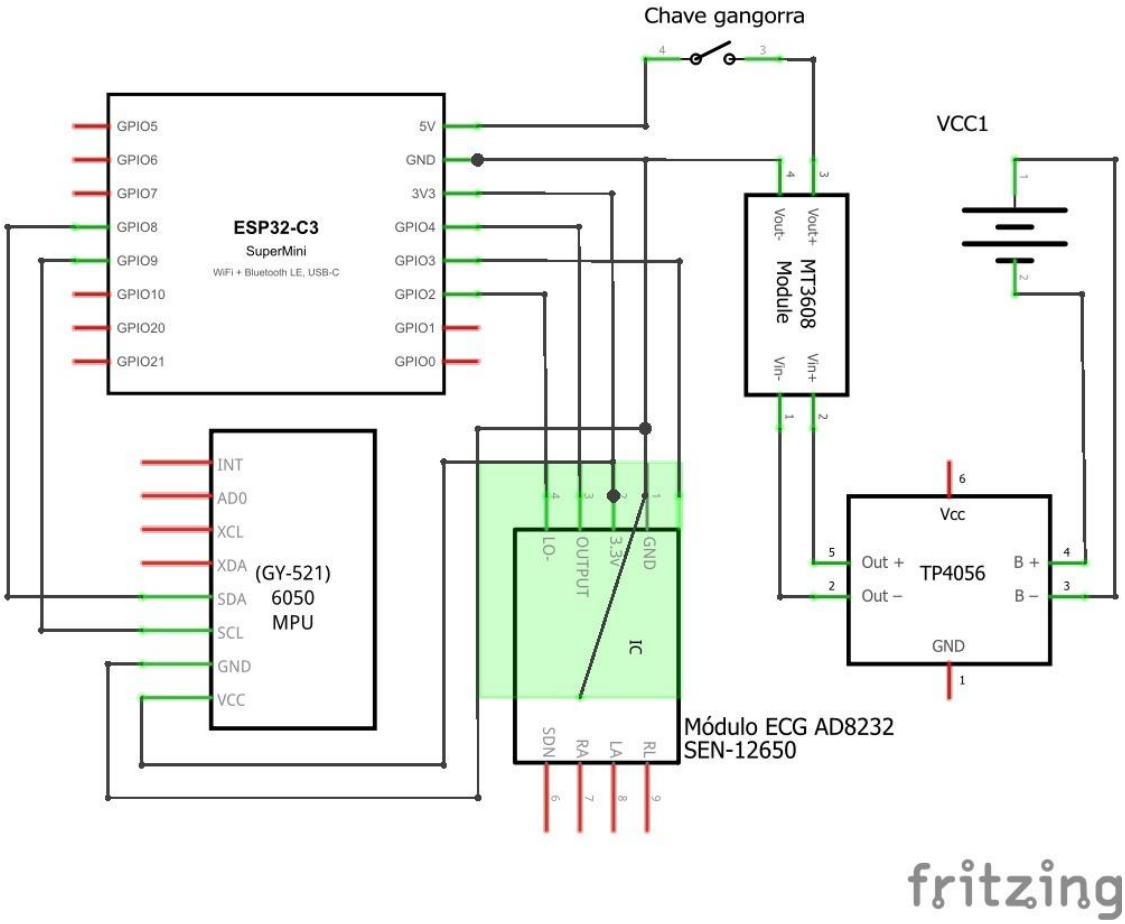
Fonte das figuras: elaborado pelos autores (2026).

Figura 4 – Disposição dos módulos no case da antena



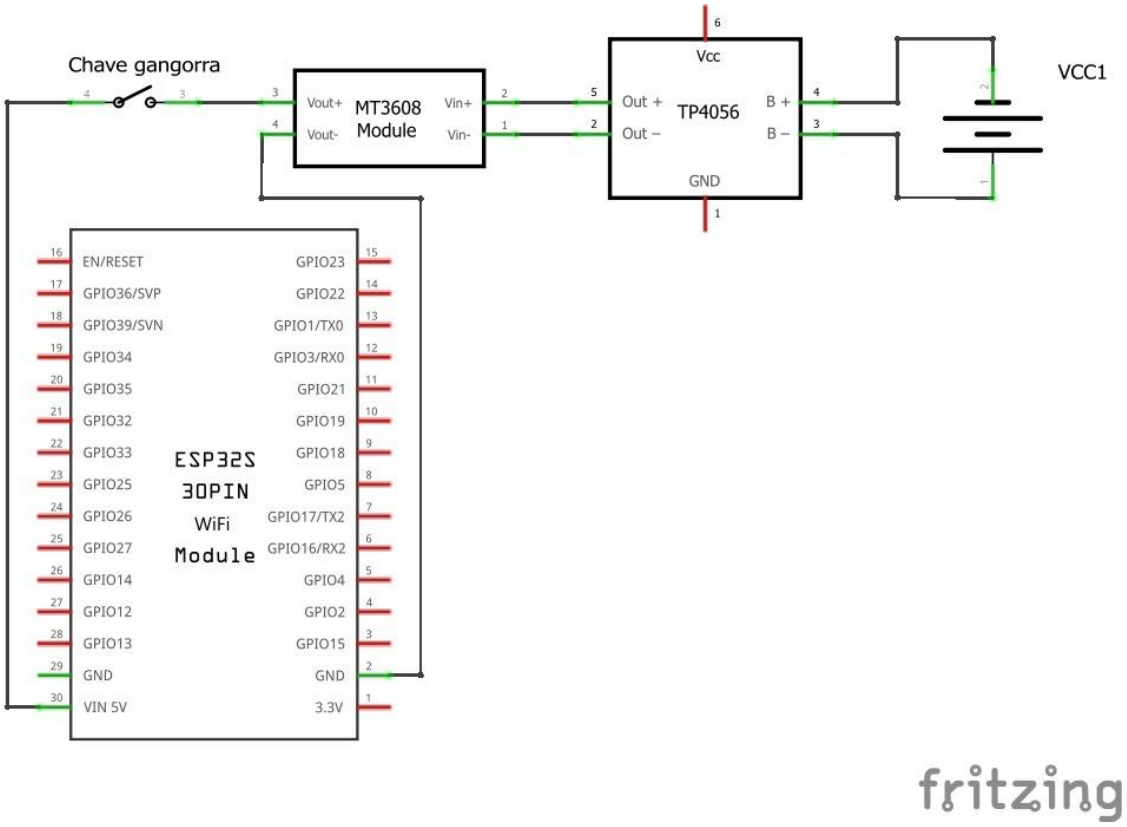
Fonte das figuras: elaborado pelos autores (2026).

Figura 5 – Esquema elétrico do cinto



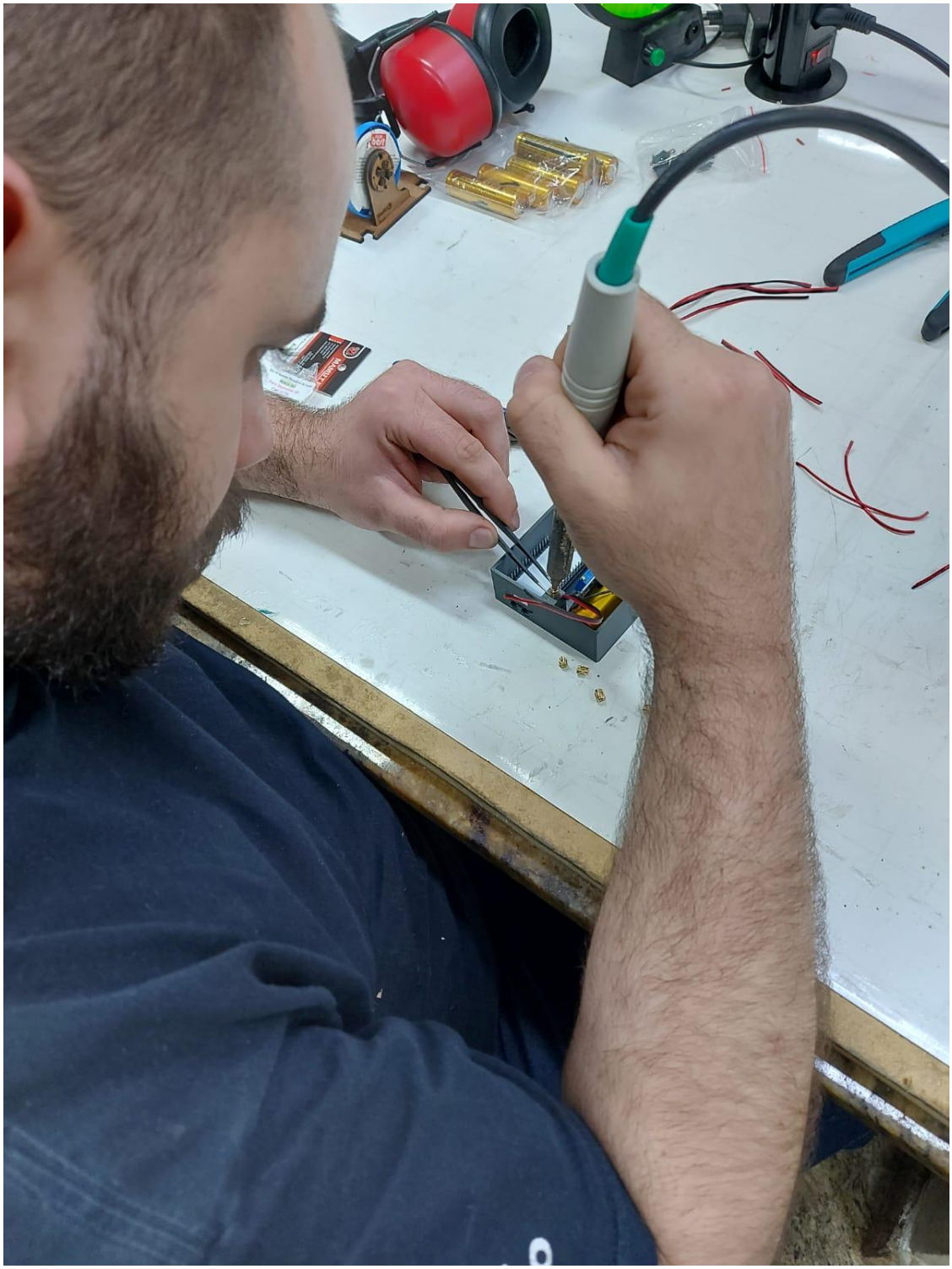
Fonte das figuras: elaborado pelos autores (2026).

Figura 6 – Esquema elétrico da antena



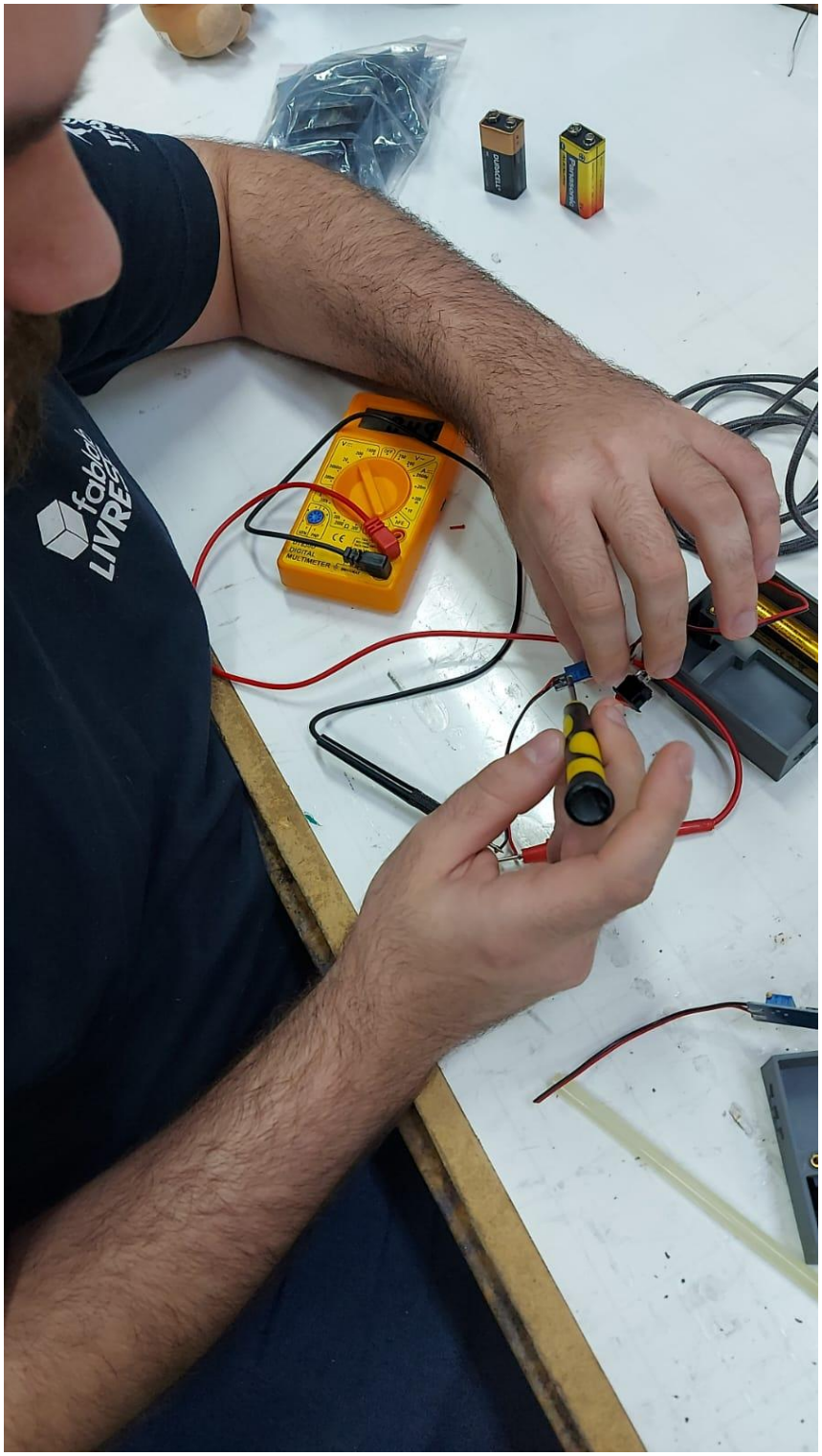
Fonte das figuras: elaborado pelos autores (2026).

Figura 7 – Registro da montagem do circuito



Fonte das figuras: elaborado pelos autores (2026).

Figura 8 – Verificação das tensões com multímetro



Fonte das figuras: elaborado pelos autores (2026).

A Figura 8 registra o uso do multímetro durante a verificação elétrica. Compare as leituras com as tensões nominais da Seção 2.4, incluindo 5,0 V na saída do MT3608 e 3,3 V no trilho que alimenta os sensores. O visor não está legível

na foto; portanto, ela documenta o procedimento de medição, mas não permite confirmar numericamente os valores obtidos.

Os arquivos editáveis dos esquemas (`cinto.fzz` e `antena.fzz`) e o instalador do Fritzing para Windows (`Instalador - fritzing - win64.msi`) estão na pasta `docs/esquemas/esquemas eletricos`. Abra os arquivos `.fzz` no Fritzing para consultar ou editar os diagramas.

Componentes passivos:

- Pull-up do I²C: o módulo GY-521 já traz resistores de pull-up em SDA e SCL; não é preciso adicionar externos.
- Desacoplamento: os módulos (ESP32-C3, ESP32 DevKit, GY-521, AD8232, MT3608 e TP4056) já trazem capacitores de desacoplamento na própria placa. O capacitor C1 (470 µF/10 V) na saída do MT3608 é recomendado para absorver os picos de corrente do rádio, sobretudo na antena.
- LO+ / LO−: saídas digitais do próprio AD8232; dispensam resistores.
- SDN do AD8232: fica aberto; o módulo o mantém em nível alto (ligado).
- R1 (330 Ω): limita a corrente do LED opcional, calculada por

$$I_{LED} = \frac{V_{GPIO} - V_{LED}}{R_1} = \frac{3{,}3 - 2{,}0}{330} \approx 4 \text{ mA}$$

em que I_{LED} é a corrente no LED (A), V_{GPIO} é a tensão do pino em nível alto (3,3 V), V_{LED} é a queda de tensão do LED (cerca de 2,0 V) e R_1 é a resistência (Ω).

2.4 Especificações de alimentação

Tensões

Tabela 1 – Tensões nominais de alimentação

Item	Cinto	Antena
Baterias	2 × 18650 em paralelo, 3,7 V nominal (3,0–4,2 V)	1 × 18650, 3,7 V nominal (3,0–4,2 V)
Saída do MT3608	5,0 V	5,0 V
Trilho de 3,3 V	Regulador linear da placa ESP32-C3 (alimenta MPU6050 e AD8232)	Regulador AMS1117 do DevKit (só o ESP32)
Carga (TP4056)	1 A, corte em 4,2 V	1 A, corte em 4,2 V

Fonte: Elaborado pelos autores (2026).

Consumo por componente (hipóteses de dimensionamento adotadas pela equipe, não medições)

Tabela 2 – Correntes adotadas como hipóteses de dimensionamento

Dispositivo	Componente	Corrente
Cinto	ESP32-C3 Super Mini	350 mA
Cinto	MPU6050 (acelerômetro + giroscópio)	5 mA
Cinto	AD8232 (ECG)	0,17 mA (170 µA)
Cinto	Total	≈ 355 mA
Antena	ESP32 DevKit V1	350 mA
Antena	Total	350 mA

Fonte: Elaborado pelos autores (2026).

Os sensores recebem 3,3 V de um regulador linear, que deixa passar a mesma corrente que recebe; por isso o total vale também para a linha de 5 V.

Caminho da energia

```
Bateria 3,7 V → TP4056 (proteção) → chave → MT3608 (sobe para 5 V) → regulador da placa (de
sce para 3,3 V) → ESP32 + sensores
```

A energia passa por duas conversões, com perdas em ambas. Para os cálculos a seguir, adota-se eficiência de 85% para o MT3608; esse valor não foi medido neste protótipo e depende da tensão, da carga e do módulo utilizado. No modelo simplificado do regulador linear, despreza-se sua corrente quiescente e considera-se que a corrente de entrada é aproximadamente igual à de saída; a diferença de tensão (5 V − 3,3 V) é dissipada como calor. Os valores de corrente adotados precisam ser confrontados com documentação do fabricante e medições na configuração real.

Por que não basta dividir capacidade por consumo: a conta direta ($\frac{2000}{355} \approx 5{,}6$ h no cinto; $\frac{1000}{350} \approx 2{,}9$ h na antena) compara a capacidade da bateria, que trabalha em 3,7 V, com a corrente do circuito, que trabalha em 5 V. Ela ignora a elevação de tensão, as perdas do conversor e a parte da carga que não pode ser usada. Por isso o cálculo é feito em potência, nos passos a seguir.

Passo 1 – Potência na saída de 5 V

$$P_{5V} = V_{5V} \cdot I_{total}$$

em que:

- P_{5V} é a potência entregue ao circuito na saída do MT3608 (W);
- V_{5V} é a tensão de saída do MT3608 (5 V);
- I_{total} é a soma das correntes dos componentes (A).

Passo 2 – Potência retirada da bateria

$$P_{bat} = \frac{P_{5V}}{\eta}$$

em que:

- P_{bat} é a potência que sai da bateria (W);
- η é a eficiência do MT3608 (0,85, ou seja, 85%).

Passo 3 – Corrente na bateria

$$I_{bat} = \frac{P_{bat}}{V_{bat}}$$

em que:

- I_{bat} é a corrente retirada da bateria (A);
- V_{bat} é a tensão da bateria (3,7 V nominal; cerca de 3,3 V quando está quase vazia).

Tabela 3 – Potências e correntes estimadas para o conjunto

Grandeza	Cinto	Antena
I_{total} (5 V)	0,355 A	0,350 A
P_{5V}	5 × 0,355 = 1,78 W	5 × 0,350 = 1,75 W
P_{bat}	1,78 / 0,85 = 2,09 W	1,75 / 0,85 = 2,06 W
I_{bat} (bateria em 3,7 V)	2,09 / 3,7 ≈ 565 mA	2,06 / 3,7 ≈ 556 mA
I_{bat} com bateria quase vazia (3,3 V)	2,09 / 3,3 ≈ 633 mA	2,06 / 3,3 ≈ 624 mA

Fonte: Elaborado pelos autores (2026).

A corrente estimada na bateria (565 mA) é maior que a do circuito (355 mA) porque a bateria tem tensão menor que a saída: para entregar a mesma potência, ela precisa fornecer mais corrente e cobrir as perdas. Sob carga de potência aproximadamente constante, a corrente tende a aumentar com a descarga. O limite da chave interna do MT3608 não deve ser interpretado como corrente contínua garantida na saída; é necessário verificar indutor, proteção, dissipação e especificação do módulo. No cinto, a divisão de cerca de 0,3 A por célula pressupõe duas células equivalentes, com tensões, capacidades e resistências internas compatíveis.

Autonomia

Passo 1 – Energia armazenada

$$E = n \cdot C \cdot V_{bat}$$

em que:

- E é a energia nominal armazenada (Wh);
- n é o número de baterias;
- C é a capacidade de cada bateria (Ah; 1000 mAh = 1,0 Ah);
- V_{bat} é a tensão nominal da bateria (3,7 V).

Passo 2 – Energia útil

$$E_{útil} = E \cdot f_u$$

em que:

- $E_{útil}$ é a energia que pode ser usada antes do corte de proteção (Wh);
- f_u é a fração útil adotada para o cálculo (0,9), não medida. A energia utilizável depende das células, da descarga e dos limiares de proteção do módulo efetivamente instalado; o corte não determina, sozinho, essa fração.

Passo 3 – Tempo de funcionamento

$$t = \frac{E_{útil}}{P_{bat}}$$

em que:

- t é a autonomia (h);
- P_{bat} é a potência retirada da bateria, calculada acima (W).

Tabela 4 – Autonomia estimada sob as hipóteses adotadas

Dispositivo	**Baterias**	**Energia nominal**	**Energia útil**	P_{bat}	**Autonomia**
Cinto	2 × 1000 mAh em paralelo	2 × 1,0 × 3,7 = 7,4 Wh	7,4 × 0,9 = 6,66 Wh	2,09 W	6,66 / 2,09 ≈ 3,19 h (3 h 11 min)
Antena	1 × 1000 mAh	1 × 1,0 × 3,7 = 3,7 Wh	3,7 × 0,9 = 3,33 Wh	2,06 W	3,33 / 2,06 ≈ 1,62 h (1 h 37 min)

Fonte: Elaborado pelos autores (2026).

A conversão para minutos usa a parte decimal: 0,19 h × 60 ≈ 11 min e 0,62 h × 60 ≈ 37 min.

Conferência em mAh: a mesma autonomia sai dividindo a capacidade útil pela corrente na bateria.

$$t = \frac{n \cdot C_{mAh} \cdot f_u}{I_{bat}}$$

em que C_{mAh} é a capacidade de cada bateria em mAh e I_{bat} é a corrente na bateria em mA. Para o cinto: 2 × 1000 × 0,9 = 1800 mAh; 1800 ÷ 565 ≈ 3,19 h. Para a antena: 1000 × 0,9 = 900 mAh; 900 ÷ 556 ≈ 1,62 h. Os dois métodos dão o mesmo resultado.

As autonomias de 3,19 h e 1,62 h são estimativas condicionadas às hipóteses adotadas, não resultados experimentais nem tempos mínimos garantidos. O consumo médio e de pico, a capacidade útil e a eficiência precisam ser medidos. A corrente de 350 mA adotada para o ESP32 não estabelece, sem referência específica, um limite superior comprovado para o conjunto.

Recarga: até validar o comportamento do módulo e seu gerenciamento de alimentação, realizar a carga com o dispositivo desligado e fora do corpo. Não recomendar operação contínua das antenas durante a carga apenas pela presença do TP4056; verificar compartilhamento de alimentação, término da carga, aquecimento e proteção na montagem real.

Tempo de recarga

O TP4056 utiliza carga em corrente constante e tensão constante (CC/CV). A corrente programada depende do resistor de configuração e das condições térmicas do módulo. Nos cálculos abaixo, adotam-se 1 A e um fator empírico de 1,2 para representar a etapa CV; não são tempos de recarga medidos. A tensão de término e o critério de encerramento devem ser conferidos na documentação do componente e no módulo instalado, sem assumir que a transição CC/CV corresponde a uma porcentagem fixa da capacidade.

$$t_{carga} \approx k \cdot \frac{n \cdot C}{I_{carga}}$$

em que:

- t_{carga} é o tempo de recarga (h);
- k é o fator da fase de tensão constante (1,2);
- n é o número de baterias;
- C é a capacidade de cada bateria (Ah);
- I_{carga} é a corrente de carga do TP4056 (1 A).

Tabela 5 – Tempos de recarga estimados com o dispositivo desligado

Dispositivo	Cálculo	Tempo de recarga
Cinto (2 × 1000 mAh)	1,2 × 2,0 Ah / 1 A	≈ 2,4 h (2 h 24 min)
Antena (1 × 1000 mAh)	1,2 × 1,0 Ah / 1 A	≈ 1,2 h (1 h 12 min)

Fonte: Elaborado pelos autores (2026).

Hipótese de recarga com o aparelho ligado, não validada: se houvesse 1 A disponível e uma carga constante de 0,565 A no mesmo nó de alimentação, a diferença seria aproximadamente 0,44 A, resultando em $1{,}2 \times \frac{2{,}0}{0{,}44} \approx 5{,}5$ h no modelo simplificado. Essa conta não demonstra o funcionamento do circuito real: a corrente varia com a tensão, e a carga pode interferir no término da recarga. O valor é mantido somente como exemplo condicionado, não como instrução operacional. Na antena, 1 A equivale a 1C para uma célula de 1000 mAh; conferir se a célula aceita essa corrente antes de programá-la.

Validação na bancada: para confirmar os valores, ligue o multímetro (escala de A) em série entre o polo positivo da bateria e o B+ do TP4056, com o dispositivo em operação normal, anote a corrente média e substitua esse I_{bat} medido na fórmula de conferência em mAh.

Dissipação térmica

TP4056 durante a carga (carregador linear):

$$P_{TP4056} = (V_{USB} - V_{bat}) \cdot I_{carga}$$

em que P_{TP4056} é a potência transformada em calor no TP4056 (W), V_{USB} é a tensão da USB (5 V), V_{bat} é a tensão da bateria naquele momento (V) e I_{carga} é a corrente de carga (1 A).

MT3608 durante o uso:

$$P_{MT3608} = P_{bat} - P_{5V}$$

em que P_{MT3608} é a perda no conversor (W), P_{bat} é a potência que sai da bateria e P_{5V} é a potência entregue em 5 V.

Regulador de 3,3 V da placa durante o uso (regulador linear):

$$P_{reg} = (V_{5V} - V_{3V3}) \cdot I_{total}$$

em que P_{reg} é a potência transformada em calor no regulador (W), V_{5V} é a tensão de entrada (5 V), V_{3V3} é a tensão de saída (3,3 V) e I_{total} é a corrente do circuito (A).

Tabela 6 – Dissipação térmica estimada dos componentes

Componente	Cálculo	Potência dissipada	Observação
TP4056 (meio da carga)	(5 − 3,7) V × 1 A	≈ 1,3 W	Estimativa sob corrente de 1 A; conferir limitação térmica do componente
TP4056 (início, bateria em 3,0 V)	(5 − 3,0) V × 1 A	≈ 2,0 W	Dissipação maior sob a mesma corrente; medir temperatura na montagem
MT3608	2,09 − 1,78 (cinto); 2,06 − 1,75 (antena)	≈ 0,31 W	Perda estimada; manter folga e verificar aquecimento
Regulador 3,3 V da placa	1,7 × 0,355 (cinto); 1,7 × 0,350 (antena)	≈ 0,60 W	Identificar o regulador e avaliar a ventilação do case

Fonte: Elaborado pelos autores (2026).

O aumento de temperatura pode ser estimado, em regime aproximadamente estacionário e sob condições compatíveis com a resistência térmica informada pelo fabricante, por

$$\Delta T = P \cdot \theta_{JA}$$

em que ΔT é o aumento de temperatura acima do ambiente (°C), P é a potência dissipada (W) e θ_{JA} é a resistência térmica junção-ambiente (°C/W). Como exemplo condicionado, θ_{JA} entre 60 e 90 °C/W e dissipação de 0,60 W resultariam em 36 a 54 °C de elevação. A faixa térmica não foi confirmada para os reguladores das placas utilizadas; o cálculo não comprova a temperatura real nem permite concluir que os 350 mA sejam necessariamente uma corrente de pico. Identificar os reguladores, consultar suas folhas de dados e medir o aquecimento no case fechado.

Para onde vai a energia do cinto: dos 2,09 W retirados da bateria, cerca de 1,17 W (3,3 V × 0,355 A, 56%) são usados pelo ESP32 e pelos sensores, 0,60 W (29%) viram calor no regulador de 3,3 V e 0,31 W (15%) viram calor no MT3608. Quase metade da energia é perdida nas duas conversões (3,7 V → 5 V → 3,3 V); uma melhoria futura seria alimentar o circuito direto em 3,3 V a partir da bateria, com um regulador de baixa queda, sem passar pelos 5 V.

Ajuste do MT3608 (obrigatório antes de ligar o ESP32): com o ESP32 desconectado, alimente o MT3608 e gire o trimpot até ler 5,0 V na saída com o multímetro. Sem esse ajuste, a saída pode passar de 5 V e danificar a placa.

Cuidados térmicos e elétricos:

- O TP4056 aquece bastante durante a carga (~1,3 W). Mantenha folga entre ele e a bateria dentro do case e não carregue o dispositivo exposto ao sol, sobre superfícies quentes ou vestido pelo usuário.
- Só ligue as duas células do cinto em paralelo se tiverem a mesma tensão (diferença < 0,1 V); do contrário, uma carrega a outra com corrente alta.
- Use apenas células 18650 de procedência conhecida e sem amassados; descarte células estufadas.

2.5 Ordem de montagem

Arquivos de impressão 3D: os projetos estão na pasta docs/impressao_3d do repositório GitHub: Case Cinto.3mf para o cinto e Case Antena.3mf para a antena. Abra o arquivo correspondente em um fatiador compatível com 3MF (por exemplo, Cura, PrusaSlicer ou Bambu Studio), confira a orientação e as configurações de impressão e envie à impressora. Para obtê-los, abra `docs/impressao_3d/` no GitHub e baixe cada arquivo, ou baixe o repositório completo pelo botão Code → Download ZIP e localize essa pasta após descompactar.

Ferramentas: ferro de solda e estanho, multímetro, alicate de corte e desencapador, tubos termorretráteis, fita dupla-face e cola quente, chave de fenda pequena (para o trimpot do MT3608 e os parafusos M3).

Regras de segurança durante a montagem:

- Solde sempre com as baterias fora do circuito; elas são as últimas a serem ligadas.
- Confira a polaridade (+ e −) com o multímetro antes de cada ligação de energia. Bateria invertida ou em curto aquece e pode estufar.
- Nunca encoste os terminais da bateria um no outro nem em ferramentas metálicas.

Cinto

1. Imprimir o case do cinto (base e tampa) e conferir se as aberturas da chave gangorra, das duas entradas USB-C e do conector P2 estão livres.

2. Instalar os insertos M3: com o ferro de solda (~220 °C), pressione as roscas de latão nos furos do case até ficarem rentes à superfície. Espere esfriar antes de rosquear.

3. Gravar o firmware antes de montar: com o ESP32-C3 solto, grave o `cinto_mvp_v1.ino` pela USB (Seção 3.2.4). Assim, se houver problema na gravação, a placa ainda está acessível.

4. Preparar as baterias: meça a tensão das duas células 18650. Elas só podem ser ligadas em paralelo se a diferença for menor que 0,1 V; se não for, carregue a de menor tensão antes.

5. Ajustar o MT3608: alimente a entrada do MT3608 com uma das baterias (ou uma fonte de 3,7 V), sem nada ligado na saída, e gire o trimpot até o multímetro marcar 5,0 V na saída.

6. Montar a alimentação (tabela “Cinto — alimentação” da Seção 2.2), ainda sem as baterias:

   - TP4056 OUT+ → chave gangorra → MT3608 VIN+;
   - TP4056 OUT− → MT3608 VIN−;
   - MT3608 VOUT+ → pino 5V do ESP32-C3; MT3608 VOUT− → GND do ESP32-C3;
   - se for usar, o capacitor C1 de 470 µF entre VOUT+ e VOUT− (terminal positivo no VOUT+).

7. Ligar as baterias: una as duas células em paralelo (positivo com positivo, negativo com negativo) e solde ao B+ e B− do TP4056.

8. Testar a alimentação: ligue a chave e meça 5,0 V no pino 5V e 3,3 V no pino 3V3 do ESP32-C3. O LED da placa deve acender. Desligue a chave para continuar.

9. Soldar o MPU6050 (tabela da Seção 2.2): VCC → 3V3, GND → GND, SDA → GPIO 8, SCL → GPIO 9. Os pinos AD0, INT, XDA e XCL ficam sem ligação.

10. Soldar o AD8232: 3.3V → 3V3, GND → GND, OUTPUT → GPIO 4, LO− → GPIO 2, LO+ → GPIO 3. O pino SDN fica sem ligação. Posicione o conector P2 alinhado com a abertura do case.

11. LED de eletrodo solto (opcional): GPIO 10 → resistor de 330 Ω → ânodo do LED; cátodo do LED → GND.

12. Teste funcional antes de fechar: ligue a chave e o painel no computador (Seção 5.1). Confira se uma antena conecta por GATT, se a taxa fica perto de 100 amostras/s, se a aceleração em repouso fica perto de 1 g e se o status do eletrodo muda ao conectar e soltar os eletrodos.

13. Isolar e fixar: cubra as soldas com termorretrátil e prenda as placas e as baterias com fita dupla-face ou cola quente. O MPU6050 deve ficar firme, sem folga. Deixe acessíveis o USB-C do TP4056, o USB-C do ESP32-C3, o conector P2 e a chave (Figura 3).

14. Fechar o case com os parafusos M3, sem apertar em excesso, e fazer o teste de carga: ligue um cabo USB-C no TP4056 e confira o LED de carga (vermelho carregando; azul ou verde quando completo).

Antena (repetir para as três unidades)

1. Imprimir o case da antena (base e tampa perfurada) e gravar ou etiquetar a identificação (`anchor01`, `anchor02` ou `anchor03`).

2. Instalar os insertos M3 como no cinto.

3. Gravar o firmware antes de montar: grave o `anchor_mvp_v1.ino` com o `ID_ANTENA` igual à etiqueta do case (Seção 3.2.3).

4. Ajustar o MT3608 para 5,0 V, como no passo 5 do cinto.

5. Montar a alimentação (tabela “Antena — ESP32 DevKit V1” da Seção 2.2), ainda sem a bateria:

    - TP4056 OUT+ → chave gangorra → MT3608 VIN+;
    - TP4056 OUT− → MT3608 VIN−;
    - MT3608 VOUT+ → pino VIN do ESP32 DevKit; MT3608 VOUT− → GND do DevKit;
    - capacitor C1 de 470 µF entre VOUT+ e VOUT− (recomendado na antena, por causa dos picos de corrente do Wi-Fi).

6. Ligar a bateria ao B+ e B− do TP4056.

7. Testar: ligue a chave e meça 5,0 V no pino VIN; o LED de alimentação do DevKit deve acender. Com o painel aberto, a antena deve aparecer como online em até 10 s.

8. Isolar, fixar e fechar: termorretrátil nas soldas, placas e bateria presas com fita dupla-face ou cola quente, Micro-USB do ESP32, USB-C do TP4056 e chave acessíveis pelas aberturas (Figura 4). Feche a tampa com os parafusos M3.

2.6 Fixação mecânica e encapsulamento

1. Disposição interna: siga a Figura 3 (cinto) e a Figura 4 (antena). Prenda as baterias e as placas com fita dupla-face ou cola quente, deixando acessíveis:
    - Cinto: USB-C do TP4056 (carga), USB-C do ESP32-C3 (gravação), conector P2 do AD8232 e a chave gangorra.
    - Antena: USB-C do TP4056, Micro-USB do ESP32 e a chave gangorra.
2. Isolamento: cubra as soldas expostas com termorretrátil e garanta que nenhum terminal de bateria encoste em partes metálicas.
3. Fechamento: feche a tampa com os parafusos M3, sem apertar em excesso.
4. Fixação do cinto: o case vai preso ao cinto na frente do corpo, na altura da cintura, posição equivalente à do dataset SisFall usado no treino. O MPU6050 deve ficar firme no case, sem folga: vibração solta altera as leituras.
5. Fixação das antenas: instale cada antena em parede ou suporte, a cerca de 1 m do chão ou mais, longe de roteadores, micro-ondas e outras fontes de 2,4 GHz, e afastadas entre si. Anote a posição (x, y) de cada uma em metros para a configuração do ambiente (Seção 5.1). As três antenas não podem ficar alinhadas (colineares).
6. Ambiente: o protótipo é para uso interno; o case não é vedado contra água. Não há partes ligadas à rede elétrica; o único aterramento é o GND comum de baixa tensão de cada dispositivo.

SEÇÃO 3 — GUIA DE INSTALAÇÃO E CONFIGURAÇÃO DO SOFTWARE

3.1 Requisitos mínimos

Obtenção do projeto: o material do sistema está organizado no repositório GitHub https://github.com/KaykyCrespo/cinto-inteligente-deteccao-quedas: código do painel e do backend, firmware do cinto e das antenas, bibliotecas Arduino usadas, modelo de IA treinado, projetos imprimíveis dos cases em `docs/impressao_3d/Case Cinto.3mf` e `docs/impressao_3d/Case Antena.3mf`, documentação e o arquivo `requirements.txt` com as dependências Python. Os registros audiovisuais de testes estão em `dados/testes/` e são apresentados na Seção 5.6. Os arquivos adicionados localmente só estarão acessíveis à banca no GitHub após sua publicação. Para baixar os projetos `.3mf` individualmente, abra docs/impressao_3d. Como alternativa, use Code → Download ZIP e localize essa pasta após descompactar, ou clone o repositório com Git:

```
git clone https://github.com/KaykyCrespo/cinto-inteligente-deteccao-quedas.git
cd cinto-inteligente-deteccao-quedas
```

Quadro 8 – Requisitos de software e conectividade

Item	Versão / requisito
Sistema operacional	Windows 10 ou 11, 64 bits
Python	3.12 ou superior (testado em 3.13)
Arduino IDE	2.3 ou superior
Pacote de placas ESP32	“esp32 by Espressif Systems” 3.x
Bibliotecas Arduino	NimBLE-Arduino 2.5.2, PubSubClient 2.8 (cópias em `Arduino/libraries/`)
Broker MQTT	Eclipse Mosquitto 2.x
Driver USB-serial	Silicon Labs CP210x (antenas); ESP32-C3 usa USB nativo (Seção 3.2.2)
Navegador	Chrome, Edge ou Firefox atualizado
Rede	Wi-Fi 2,4 GHz com o computador e as antenas na mesma sub-rede

Fonte: Elaborado pelos autores (2026).

Dependências Python (`requirements.txt`):

Quadro 9 – Dependências Python e respectivas funções

Pacote	Versão	Uso
streamlit	1.52.2	Painel web
paho-mqtt	2.1.0	Cliente MQTT
pandas	2.3.3	Tabelas e exportação
numpy	2.2.0	Cálculo dos atributos
scipy	1.14.1	Reamostragem no treino
scikit-learn	1.9.1	Random Forest
joblib	1.4.2	Carga do modelo
altair	6.0.0	Gráficos

Fonte: Elaborado pelos autores (2026).

3.2 Configuração da borda (firmware)

3.2.1 Instalar a Arduino IDE e o pacote ESP32

1. Baixe e instale a Arduino IDE 2.x em https://www.arduino.cc/en/software.
2. Em Arquivo → Preferências → URLs adicionais para Gerenciadores de Placas, adicione:

   `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
3. Em Ferramentas → Placa → Gerenciador de Placas, pesquise `esp32` e instale esp32 by Espressif Systems (3.x).
4. Bibliotecas: copie as pastas de `Arduino/libraries/` do repositório para `Documentos\Arduino\libraries\`. Outra opção é instalar pelo Gerenciador de Bibliotecas `NimBLE-Arduino` (h2zero) 2.5.2 e `PubSubClient` (Nick O'Leary) 2.8. Reinicie a IDE.

3.2.2 Drivers USB e reconhecimento da porta COM

Quadro 10 – Interfaces USB e drivers das placas

Placa	Interface USB	Driver no Windows
Antena — ESP32 DevKit V1	Chip CP2102 (Silicon Labs)	Instale o CP210x Universal Windows Driver da página "CP210x USB to UART Bridge VCP Drivers" da Silicon Labs (descompacte, clique com o botão direito em `silabser.inf` → Instalar)
Antena — clone com chip CH340	CH340/CH341	Driver CH341SER da WCH
Cinto — ESP32-C3 Super Mini	USB nativo (USB Serial/JTAG)	Nenhum: o Windows 10/11 reconhece sozinho

Fonte: Elaborado pelos autores (2026).

Para conferir:

1. Conecte a placa com um cabo USB de dados (cabos só de carga não aparecem).
2. Abra o Gerenciador de Dispositivos → Portas (COM e LPT):
   - Antena: `Silicon Labs CP210x USB to UART Bridge (COMx).`
   - Cinto: `Dispositivo Serial USB (COMx).`
3. Se surgir um "Dispositivo desconhecido" com triângulo amarelo, instale o driver da tabela e reconecte.

4. Se o ESP32-C3 não aparecer: segure BOOT, conecte o cabo (ou aperte RESET) e solte BOOT. A placa entra em modo de gravação e a porta aparece.

Atenção: antes de conectar o USB para gravar, desligue a chave gangorra. Assim o 5 V do MT3608 não fica em paralelo com o 5 V do USB.

3.2.3 Configurar e gravar o firmware da antena (`Arduino/anchor_mvp_v1/anchor_mvp_v1.ino`)

1. Configure o Wi-Fi em um arquivo local: na pasta `Arduino/anchor_mvp_v1/`, copie `credenciais_exemplo.h` para `credenciais.h` e preencha com a rede 2,4 GHz:

```
#define WIFI_SSID       "<nome da rede 2,4 GHz>"
#define WIFI_PASSWORD   "<senha da rede>"
```

O `credenciais.h` é ignorado pelo git, então a senha não vai para o repositório. Se ele não existir, o sketch compila com valores de exemplo e a antena não conecta ao Wi-Fi.

2. Abra o sketch e confira o topo do arquivo, ajustando o número da antena e o IP do broker:

```
#define CODIGO_AMBIENTE "OQPNB"          // igual ao do cinto
#define ID_ANTENA       1                // 1, 2 ou 3 (um por placa; igual à etiquet
a do case)
#define MQTT_BROKER     "<IPv4 do computador central>"
#define MQTT_PORT       1883
#define MQTT_USER       ""               // vazio: broker sem autenticação (rede loc
al)
#define MQTT_PASS       ""
#define MQTT_TOPIC_BASE "oqpnb/anchor"
```

O IPv4 do computador aparece com `ipconfig` no PowerShell (linha Endereço IPv4 do adaptador Wi-Fi).

Importante: o computador (notebook) que roda o broker e o painel precisa estar conectado à mesma rede Wi-Fi configurada em `credenciais.h`. Em outra rede, as antenas não alcançam o broker e o painel mostra “Antena online, mas nenhum pacote de cinto chegou ainda. Verifique no Serial da antena se ela mostra ‘GATT: CONECTADO’ e ‘TX MQTT’ > 0.”, mesmo com o cinto ligado. Se mudar de rede, ajuste `credenciais.h` e `MQTT_BROKER` e grave o firmware de novo.

3. Ferramentas → Placa → esp32 → ESP32 Dev Module (ou DOIT ESP32 DEVKIT V1).
4. Ferramentas → Porta → a COM da antena.

5. Clique em Carregar. Se aparecer `Connecting....___`, segure BOOT até começar a gravar.
6. Abra o Monitor Serial a 115200 baud. A cada 5 s a antena imprime o modo (`GATT+SENSORES` ou `SCAN_RSSI`), RSSI, contadores e estado do Wi-Fi/MQTT.
7. Repita para as antenas 2 e 3, mudando apenas `ID_ANTENA`.

Tópicos MQTT publicados pela antena:

Quadro 11 – Tópicos MQTT e conteúdo das mensagens

Tópico	Conteúdo
`oqpnb/anchor/<ID_ANTENA>`	`{OQPNB;id_antena;rssi;id_cinto;ax;ay;az;gx;gy;gz;eletrodos;ecg}` (com GATT) ou `{OQPNB;id_antena;rssi;id_cinto;;;;;;;;}` (só RSSI)
`oqpnb/anchor/<ID_ANTENA>/status`	`online` / `offline` (mensagem retida, com Last Will)

Fonte: Elaborado pelos autores (2026).

3.2.4 Configurar e gravar o firmware do cinto (`Arduino/cinto_mvp_v1/cinto_mvp_v1.ino`)

1. Confira o topo do arquivo:

```
#define CODIGO_AMBIENTE "OQPNB"   // igual ao das antenas
#define ID_CINTO        1
#define FREQ_LEITURA_HZ 100       // deve ser igual à do modelo
```

Não altere as faixas do MPU em `iniciarMPU()` (±8 g / ±2000 °/s): o modelo foi treinado nelas.

2. Ferramentas → Placa → esp32 → ESP32C3 Dev Module.
3. Ferramentas → USB CDC On Boot → Enabled (sem isso o Monitor Serial fica vazio).
4. Selecione a porta e clique em Carregar.
5. No Monitor Serial (115200), o cinto informa quando uma antena conecta, quando o código de ambiente é validado e quando o anúncio alterna entre conectável e beacon.

Comunicação BLE do cinto:

Quadro 12 – Serviço GATT e anúncios BLE do cinto

Elemento	Valor
Serviço GATT	`FFF0`
`FFF1` (write)	A antena escreve o `CODIGO_AMBIENTE`; o cinto só transmite depois de validar (prazo de 5 s)
`FFF2` (notify)	Pacote a 100 Hz: `{OQPNB;id_cinto;ax;ay;az;gx;gy;gz;eletrodos;sinal_ad8232}`
Advertising	Manufacturer Data `{OQPNB;id_cinto}`; conectável a cada 20–40 ms sem GATT; beacon não conectável a cada 100 ms com GATT ativo

Fonte: Elaborado pelos autores (2026).

Uma antena por vez se conecta por GATT e repassa os sensores; as outras medem o RSSI dos anúncios.

3.3 Subida do servidor, backend e banco de dados

3.3.1 Broker Mosquitto

1. Instale o Mosquitto 2.x para Windows (https://mosquitto.org/download/), mantendo o serviço do Windows marcado.
2. Edite `C:\Program Files\mosquitto\mosquitto.conf` (Bloco de Notas como administrador) e adicione:

```
listener 1883 0.0.0.0
allow_anonymous true
```

Configuração para rede local de laboratório. Em produção, use `allow_anonymous false` com `password_file` e TLS na porta 8883.

3. Em um PowerShell como administrador:

```
Restart-Service mosquitto
New-NetFirewallRule -DisplayName "Mosquitto MQTT 1883" -Direction Inbound -Protocol TCP -LocalPort 1883 -Action Allow -Profile Private,Domain
```

4. Em Configurações → Rede e Internet → Wi-Fi → (rede), marque a rede como Privada.

3.3.2 Backend Python

O repositório já traz o arquivo `requirements.txt`, que lista todas as bibliotecas Python do sistema com as versões exatas (tabela da Seção 3.1). Não é preciso instalar uma por uma: um único comando instala todas as dependências.

1. Instale o Python 3.12 ou superior em https://www.python.org/downloads/. Na primeira tela do instalador, marque Add python.exe to PATH.
2. Abra o PowerShell na pasta do projeto (no Explorador de Arquivos, abra a pasta, clique na barra de endereço, digite `powershell` e pressione Enter).
3. Crie e ative um ambiente virtual (recomendado, para não misturar com outras instalações do computador):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Se o PowerShell bloquear a ativação, rode antes `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` e confirme.

4. Instale as dependências com um único comando:

```
pip install -r requirements.txt
```

O `pip` lê o `requirements.txt` e baixa todas as bibliotecas nas versões indicadas.

5. Valide a instalação rodando os testes automatizados; o resultado esperado é `OK`:

```
python -m unittest discover -s tests -p "teste_*.py" -v
```

O arquivo `requirements-docs.txt` traz dependências opcionais, usadas apenas para regenerar os diagramas e este manual (`pip install -r requirements-docs.txt`); não é necessário para operar o sistema.

3.3.3 Banco de dados

- Usa SQLite, sem servidor a instalar. O arquivo `dados/oqpnb.db` e as 7 tabelas (`cinto`, `antena`, `sessao`, `amostra`, `janela`, `queda`, `status_antena`) são criados automaticamente por `banco.py` (`CREATE TABLE IF NOT EXISTS`) na primeira abertura do painel. Não há migração manual.
- O banco funciona em modo WAL e grava em lotes por uma thread dedicada. Os horários ficam em UTC (ISO-8601).
- O DER, os modelos lógico e físico e o dicionário de dados estão em `dados/README_BANCO_DE_DADOS.md`.

3.3.4 Variáveis de ambiente

O painel lê as variáveis de ambiente do processo. O exemplo abaixo reúne os parâmetros utilizados; o arquivo fornecido para credenciais locais é `segredos.env.exemplo`:

```
# Telegram (obrigatórias para enviar alertas)
OQPNB_TG_TOKEN=<token gerado pelo @BotFather>
OQPNB_TG_CHAT=<chat_id do destinatário>

# MQTT (padrão: localhost:1883)
OQPNB_BROKER=localhost
OQPNB_PORT=1883

# Caminhos (padrão: pastas do projeto)
OQPNB_MODELO=modelos/modelo_mvp_8g.joblib
OQPNB_DB=dados/oqpnb.db

# Localização (valores iniciais do software; calibração não validada)
OQPNB_LARGURA_M=5.0
OQPNB_ALTURA_M=5.0
OQPNB_RSSI_1M=-62
```

```
OQPNB_N_PERDA=2.0
OQPNB_OFFSET_GATT=0
```

Para definir no PowerShell, na mesma janela que abrirá o painel:

```
$env:OQPNB_TG_TOKEN = "<token>"
$env:OQPNB_TG_CHAT  = "<chat_id>"
```

Credenciais fora do repositório: o token do bot, o chat ID e a senha do Wi-Fi nunca ficam no código versionado. O token e o chat ID podem ser definidos pelas variáveis de ambiente acima ou no arquivo local `segredos.env`, na raiz do projeto: copie `segredos.env.exemplo` para `segredos.env` e preencha `OQPNB_TG_TOKEN` e `OQPNB_TG_CHAT`. Esse arquivo é ignorado pelo git e lido automaticamente ao abrir o painel; variáveis já definidas no ambiente têm prioridade sobre ele. A senha do Wi-Fi das antenas fica em `credenciais.h` (Seção 3.2.3). Em nenhuma hipótese publique tokens e senhas reais em repositórios ou arquivos versionados.

Como criar o bot do Telegram e obter o token e o chat ID

O alerta de queda é enviado por um bot do Telegram. São necessários dois dados: o token (a chave de acesso do bot à API do Telegram) e o chat ID (o identificador da conversa que vai receber os alertas).

Parte A – Criar o bot e obter o token:

1. Instale o Telegram no celular ou no computador e entre com a sua conta.
2. Na busca, procure por @BotFather (conta oficial do Telegram, com selo azul de verificação) e abra a conversa.
3. Toque em Iniciar (ou envie `/start`) e depois envie o comando `/newbot`.
4. O BotFather pede o nome de exibição do bot. Envie, por exemplo, `Alerta Cinto Inteligente`.
5. Em seguida, ele pede o nome de usuário do bot, que deve ser único e terminar em `bot`. Envie, por exemplo, `cinto_inteligente_alerta_bot`. Se o nome já existir, tente outro.
6. O BotFather responde com a mensagem *“Use this token to access the HTTP API”*, seguida do token, no formato `123456789:ABCdefGhIJKlmNoPQRstuVWxyz`. Copie e guarde esse valor: ele é o `OQPNB_TG_TOKEN`.

7. Trate o token como uma senha. Se ele for exposto, gere um novo no BotFather com o comando `/revoke`, escolha o bot e atualize a variável.

Parte B – Obter o chat ID:

1. Na mesma resposta, o BotFather envia o link do bot (`t.me/<nome_de_usuário>`). Abra o link, toque em Iniciar e envie qualquer mensagem, por exemplo `oi`. Esse passo é obrigatório: um bot não pode iniciar uma conversa, só responder a quem já falou com ele.
2. No navegador, acesse o endereço abaixo, trocando `<TOKEN>` pelo token da Parte A (sem os sinais `<` e `>`):

```
https://api.telegram.org/bot<TOKEN>/getUpdates
```

3. A página mostra um texto em formato JSON. Procure o trecho `"chat":{"id":123456789, ...}`. O número após `"id":` é o chat ID, que vai em `OQPNB_TG_CHAT`.
4. Se aparecer apenas `{"ok":true,"result":[]}`, a mensagem ainda não chegou: envie outra mensagem ao bot e recarregue a página.
5. Para enviar os alertas a um grupo (por exemplo, toda a equipe de cuidado): crie o grupo, adicione o bot como membro, envie uma mensagem no grupo mencionando o bot (`@cinto_inteligente_alerta_bot oi`) e repita os passos 2 e 3. O chat ID de grupo é negativo (por exemplo, `-1001234567890`) e deve ser usado com o sinal de menos.

Parte C – Configurar e testar:

1. Copie `segredos.env.exemplo` para `segredos.env`, na raiz do projeto, e preencha:

```
OQPNB_TG_TOKEN=<token da Parte A>
OQPNB_TG_CHAT=<chat ID da Parte B>
```

Outra opção é definir as variáveis no PowerShell que vai abrir o painel (`$env:OQPNB_TG_TOKEN = "<token>"` e `$env:OQPNB_TG_CHAT = "<chat ID>"`) ou de forma permanente com `setx`, que só vale para as janelas abertas depois do comando.

2. Abra o painel (`python -m streamlit run app.py`). Na barra lateral, a opção Enviar alerta de quedas fica habilitada.
3. Clique em Testar envio. O bot deve enviar a mensagem "☑ Teste do Cinto Inteligente" com a data e a hora. Se aparecer erro, consulte a Seção 5.3 (erros 401, 403 e 404 do Telegram).

3.4 Aplicação cliente (painel)

```
python -m streamlit run app.py
```

O painel abre em http://localhost:8501. Outros computadores da mesma rede acessam pelo `http://<IP do computador>:8501`.

Ferramentas auxiliares (opcionais):

```
# monitor em terminal
python interface.py --modelo modelos/modelo_mvp_8g.joblib --quiet

# reprocessar uma gravação sem hardware
python scripts/reproduzir_csv.py dados/gravacoes/<arquivo>.csv --modelo modelos/modelo_mvp_
8g.joblib
```

SEÇÃO 4 — MÓDULO DE INTELIGÊNCIA ARTIFICIAL

4.1 Objetivo operacional

A IA realiza classificação binária supervisionada de janelas de movimento: para cada janela de 2,5 s do sensor inercial, retorna uma estimativa de probabilidade da classe queda, usada para indicar possível queda em relação a atividade sem queda. Esse valor não foi validado como probabilidade calibrada de risco clínico. Quando o critério de decisão é atendido (limiar padrão 0,50, ajustável no painel), o sistema registra um evento de possível queda e apresenta o alerta; o envio pelo Telegram depende de habilitação, credenciais e conectividade. A localização por RSSI e o ECG não integram o classificador e são informações contextuais.

4.2 Dataset de treinamento e validação

Quadro 13 – Caracterização do conjunto SisFall utilizado

Item	Descrição
Origem	Dataset público SisFall (Sucerquia; López; Vargas-Bonilla, 2017), em `dados/SISFALL/`
Participantes	38 voluntários (23 adultos jovens SA01–SA23 e 15 idosos SE01–SE15)
Registros	1.798 quedas simuladas (F01–F15) e 2.707 atividades da vida diária (D01–D19)
Sensor usado	Acelerômetro ADXL345 (±16 g) e giroscópio ITG3200 (±2000 °/s) presos à cintura, a 200 Hz
Volume após janelamento	122.572 janelas (28.768 de queda e 93.804 de atividade)

Fonte: Elaborado pelos autores (2026).

Pré-processamento (`scripts/treino_sisfall.py`):

1. Conversão das contagens do SisFall para g e °/s.
2. Reamostragem de 200 Hz para 100 Hz, a taxa do cinto.
3. Simulação do sensor do cinto: saturação em ±8 g / ±2000 °/s, as mesmas faixas configuradas no MPU6050.
4. Janelamento: janelas de 2,5 s com passo de 0,5 s. Nos registros de queda, entram como positivas as janelas que contêm o impacto.

Engenharia de atributos (`features.py`, a mesma função no treino e na operação). Magnitudes e o ângulo entre vetores são invariantes a uma rotação fixa ideal dos eixos; isso não elimina offsets, ruído ou artefatos de movimento. O Random Forest utilizado não exige escalonamento dos atributos para seus

particionamentos por limiar, questão distinta da calibração física do sensor. A interpretação de `tilt` como mudança da direção da gravidade pressupõe trechos em que a aceleração dinâmica não predomine:

Tabela 7 – Atributos calculados e importâncias no modelo final

Atributo	Definição	Importância no modelo
`tilt`	Ângulo entre os vetores médios de aceleração no início e no fim da janela (graus)	0,33
`gstd`	Desvio padrão de |ω| (°/s)	0,18
`amax`	Máximo de |a| (g)	0,13
`gmax`	Máximo de |ω| (°/s)	0,12
`astd`	Desvio padrão de |a| (g)	0,08
`jerk`	Máxima variação de |a| por segundo (g/s)	0,07
`post_std`	Desvio padrão de |a| nos últimos 0,5 s; descreve variabilidade, não comprova imobilidade após impacto	0,04
`amin`	Mínimo de |a| na janela; não comprova, isoladamente, queda livre	0,04

Fonte: Elaborado pelos autores (2026).

4.3 Arquitetura do modelo e treinamento

Algoritmo: Random Forest (`sklearn.ensemble.RandomForestClassifier`).

Justificativa da escolha e limites da comparação:

- Regra de limiar (pico de aceleração): alternativa simples a ser avaliada sob a mesma divisão por voluntário. Não foram apresentados neste manual resultados desse classificador; não se afirma superioridade empírica do Random Forest em relação a ele.
- Redes neurais (CNN/LSTM): alternativas para modelar padrões temporais, não implementadas nem comparadas neste experimento. Seu custo e desempenho dependem da arquitetura, dos dados e do hardware; não se presume inferioridade ou exigência universal de mais recursos.
- Random Forest: foi escolhido para trabalhar com os atributos tabulares definidos pela equipe e disponibilizar importância por redução de impureza (Breiman, 2001; Pedregosa et al., 2011). Os resultados deste modelo estão na Seção 4.4. O tempo de inferência no computador de operação não foi medido neste manual; não se garante processamento em milissegundos.

A comparação experimental com uma regra de limiar, prevista nos objetivos da parte escrita, permanece pendente de registro dos resultados. Sua ausência limita conclusões sobre a vantagem relativa do algoritmo, mas não altera os valores medidos para o Random Forest.

Hiperparâmetros:

Quadro 14 – Configuração do Random Forest

Parâmetro	Valor	Motivo
`n_estimators`	200	Estabilidade da votação
`min_samples_leaf`	2	Reduz o sobreajuste
`class_weight`	`balanced_subsample`	Compensa o desbalanceamento (23% de janelas de queda)
Limiar de decisão	0,50	Equilíbrio entre sensibilidade e especificidade (Seção 4.4)

Fonte: Elaborado pelos autores (2026).

Validação: validação cruzada por voluntário (GroupKFold, 5 folds). Nenhum voluntário aparece ao mesmo tempo no treino e no teste. Isso evita o vazamento causado por janelas sobrepostas da mesma pessoa e simula o uso por alguém que o modelo nunca viu. O modelo final é treinado com todos os dados e salvo em `modelos/modelo_mvp_8g.joblib`.

Para retreinar:

```
python scripts/treino_sisfall.py --sisfall dados/SISFALL/SisFall_dataset --saida modelos/mo
delo_mvp_8g.joblib
```

4.4 Resultados e métricas

As métricas são calculadas por ensaio, com validação cruzada por voluntário (GroupKFold, 5 folds), mantendo os voluntários de teste separados dos de treinamento. Cada arquivo de gravação do SisFall corresponde a um ensaio. Um ensaio é classificado como queda se ao menos uma de suas janelas tiver probabilidade maior ou igual ao limiar: isso conta como verdadeiro positivo em um ensaio de queda e como falso positivo em um ensaio de atividade sem queda. O relatório completo está em `modelos/resultado_treino_mvp_8g.txt`.

Esse critério considera a maior probabilidade ao longo de todo o ensaio. Portanto, não comprova que o alarme ocorreu no instante da queda, não mede o atraso de detecção e não representa uma taxa de falsos alarmes por hora de uso contínuo. As métricas apresentadas são calculadas com as contagens agregadas dos folds, não pela média simples das métricas de cada fold.

Matriz de confusão (validação cruzada por voluntário, limiar 0,50):

Tabela 8 – Matriz de confusão por ensaio, com limiar 0,50

	Previsto: queda	**Previsto: atividade**
Real: queda (1.798)	VP = 1.777	FN = 21
Real: atividade (2.707)	FP = 128	VN = 2.579

Fonte: Elaborado pelos autores (2026).

Tabela 9 – Métricas agregadas da validação por voluntário

Métrica	**Fórmula**	**Valor**
Revocação (sensibilidade)	VP / (VP + FN)	98,8%
Especificidade	VN / (VN + FP)	95,3%
Precisão	VP / (VP + FP)	93,3%
F1-score	2·P·R / (P + R)	96,0%
Acurácia	(VP + VN) / total	96,7%

Fonte: Elaborado pelos autores (2026).

em que VP (verdadeiros positivos) são ensaios de queda classificados como queda; FN (falsos negativos) são ensaios de queda não detectados; FP (falsos positivos) são ensaios de atividades sem queda classificados como queda; VN (verdadeiros negativos) são ensaios de atividades sem queda corretamente classificados; P é a precisão; R é a revocação; e total é a soma VP + FN + FP + VN (4.505 ensaios).

Sensibilidade por fold: 96,8% · 98,1% · 100% · 100% · 99,5%. Especificidade por fold: 96,3% · 94,8% · 97,4% · 94,2% · 93,6%.

Efeito do limiar (configurável no painel):

Tabela 10 – Sensibilidade e especificidade por limiar de decisão

Limiar	**Sensibilidade**	**Especificidade**
0,3	99,2%	91,9%
0,4	99,0%	94,0%
0,5	98,8%	95,3%
0,6	98,4%	96,5%
0,7	97,9%	97,8%
0,8	97,0%	98,6%

Fonte: Elaborado pelos autores (2026).

Limitações conhecidas:

- A importância dos atributos mostrada em `docs/esquemas/metricas_modelo.png` vem do modelo final treinado em todos os dados, pela redução média de impureza. Não é uma métrica de validação cruzada nem demonstra efeito causal; os valores foram arredondados no relatório e sua soma apresentada pode diferir de 1.
- A maior fonte de falsos alarmes é a atividade D11 (tentar levantar e desabar na cadeira: 53/190), seguida de D06 (subir e descer escadas rápido),

D14 (mudar de posição deitado), D18 (tropeçar) e D19 (saltar). São movimentos de fato parecidos com uma queda.

- As quedas do SisFall foram simuladas, em sua maioria por adultos jovens; o desempenho com idosos reais em uso contínuo não foi medido.
- O modelo não aprende em operação; incorporar novos dados exige retreino.
- Após um alarme, o mesmo cinto fica 5 s sem gerar novo alarme, para evitar alertas duplicados.

4.5 Declaração formal de uso de IA assistiva

Foram usadas ferramentas de IA generativa como apoio à documentação, à diagramação e ao código. A tabela preserva as informações declaradas pela equipe na parte escrita, sem tratá-las como comprovação independente. Os nomes de versões e percentuais devem ser confrontados com os registros de uso antes da entrega. Nenhuma ferramenta é autora ou coautora do trabalho; a responsabilidade pela revisão, validação e conteúdo final é dos integrantes. A declaração de ausência de dados pessoais ou sensíveis enviados às ferramentas também depende da conferência dos registros pela equipe.

Quadro 15 – Uso declarado de ferramentas de IA assistiva e pendências de comprovação

Ferramenta	Versão e ambiente	Período	Finalidade	Intervenção humana
Claude (Anthropic)	Denominação informada: claude-opus-5-5; versão não confirmada	14/09/2026 a 12/11/2026, conforme declaração da equipe	Organização do material, revisão e diagramas	Correção de 80% informada pela equipe; método de cálculo e registros não apresentados neste manual
Claude (Anthropic)	Denominação informada: Sonnet 4.6 low; versão não confirmada	14/09/2026 a 12/11/2026, conforme declaração da equipe	Desenvolvimento web e firmware	Refatoração de cerca de 75% informada; método de cálculo e registros não apresentados neste manual
ChatGPT (OpenAI)	Denominação informada: ChatGPT 5.6 Sol extra alto; versão não confirmada	14/10/2026 a 12/11/2026, conforme declaração da equipe	Desenvolvimento web e firmware	Refatoração de cerca de 75% informada; método de cálculo e registros não apresentados neste manual
GitHub Copilot	Assistente no VS Code; modelo não identificado nesta revisão	12/11/2026, revisão deste manual	Diagnóstico técnico, revisão textual e ajustes no gerador de documentos	Alterações sujeitas à aprovação e à revisão final dos integrantes; nenhum percentual de intervenção foi calculado

Fonte: Elaborado pelos autores (2026).

A equipe informa manter registros de prompts e saídas. A existência, a disponibilidade e a integridade desse arquivo precisam ser confirmadas antes de afirmar rastreabilidade completa. Conferir também a formalização e a assinatura da declaração no Apêndice A do TCC. Percentuais sem método verificável não devem ser apresentados como indicadores auditados.

O modelo de Random Forest do produto não é IA generativa: foi treinado pela equipe com o script `scripts/treino_sisfall.py` sobre o dataset público SisFall.

SEÇÃO 5 — MANUAL DE OPERAÇÃO E DIAGNÓSTICO

5.1 Partida e calibração

Ordem de partida:

1. Computador: confirme que o serviço Mosquitto está em execução (`Get-Service mosquitto → Running`) e que o Wi-Fi do computador está conectado à mesma rede configurada nas antenas (`credenciais.h`). Em outra rede, nenhum dado do cinto chega ao painel.

2. Painel: abra o PowerShell na pasta do projeto, ative o ambiente virtual, defina as variáveis do Telegram e rode `python -m streamlit run app.py`.

3. Antenas: ligue a chave gangorra das três antenas. Em até ~10 s, cada uma deve aparecer como online na aba Detalhes → Status do Sistema.

4. Eletrodos: coloque os três eletrodos (RA, LA e RL) na pele limpa e seca e encaixe o cabo P2 no cinto.

5. Cinto: ligue a chave gangorra. Uma antena se conecta por GATT e o cartão do cinto aparece na aba Monitoramento com taxa próxima de 100 amostras/s.

Verificação do handshake:

Quadro 16 – Verificações de comunicação e aquisição na partida

Ponto	Onde verificar	Esperado
Antena ↔ Wi-Fi/MQTT	Monitor Serial da antena; painel → Detalhes	Linha `[STATUS] ... WiFi: ...` sem falhas e `TX MQTT` crescendo; antena online
Cinto ↔ antena (GATT)	Monitor Serial da antena; painel → Detalhes → antenas	`[GATT] CONECTADO E AUTENTICADO!` e modo `GATT+SENSORES` em uma antena
Código de ambiente	Monitor Serial do cinto	Código validado
Fluxo de dados	Painel → Monitoramento	Amostras subindo, ~100 Hz
Eletrodos	Painel → Status Eletrodo; LED do cinto	“Conectado”; LED apagado

Fonte: Elaborado pelos autores (2026).

Calibração de repouso dos sensores:

1. Com o cinto parado sobre a mesa por 10 s, abra Detalhes → Gráficos de Sinais. A magnitude da aceleração deve ficar próxima de 1 g (gravidade) e o giroscópio próximo de 0 °/s.

2. Magnitudes não corrigem offsets físicos (Seção 4.2). Se o repouso se afastar de 1 g ou o giroscópio não se aproximar de 0 °/s, verificar a fixação, as faixas, as condições de medição e a necessidade de calibração do sensor. Não modificar o pré-processamento usado pelo modelo sem validar a compatibilidade entre treino e operação.

3. ECG: com o usuário sentado e parado, o traçado deve mostrar complexos QRS regulares. O ECG é experimental e sem finalidade médica.

Configuração do ambiente (localização):

1. Na aba Detalhes, abra o expansor Ambiente ao lado do mapa.

2. Informe a largura e a altura do cômodo em metros e as coordenadas (x, y) de cada antena, medidas a partir do canto inferior esquerdo.

3. Os valores ficam salvos em `dados/ambiente.json`. Se as antenas estiverem alinhadas, o painel avisa: reposicione-as.

Calibração do RSSI (recomendada a cada novo ambiente):

1. Coloque o cinto, vestido, a 1 m de uma antena, de frente para ela.

2. Na barra lateral, clique em Gravar amostras por ~60 s e depois em Parar gravação.

3. Separar os registros dessa antena por origem (GATT ou Advertising) e calcular a mediana por segundo antes da mediana do ensaio, reduzindo o peso de RSSIs repetidos nas linhas de sensores. Não preencher lacunas. Registrar altura, orientação, movimento, obstáculos e distância real. O valor é um candidato a $RSSI_{1m}$, não uma referência validada automaticamente.

4. Repetir a 2, 3, 5 e 7 m, mantendo a mesma antena, altura, orientação e modo de recepção, com repetições e posições de referência. Como aproximação entre dois pontos, estimar:

$$n_{perda} = \frac{RSSI_{1m} - RSSI_d}{10\log_{10} d}$$

em que n_{perda} é o expoente de perda (adimensional), $RSSI_{1m}$ e $RSSI_d$ são as medianas dos ensaios (dBm) e d é a distância real entre o cinto e a antena, em metros ($d > 1$). A fórmula corresponde a $\log_{10}(d/1\,\mathrm{m})$ e não valida o modelo por si só. Preferir ajuste com todos os pontos e examinar resíduos, repetibilidade e erro de localização contra posições conhecidas.

5. Somente adotar `OQPNB_RSSI_1M` e `OQPNB_N_PERDA` se os ensaios controlados sustentarem a relação distância-RSSI e melhorarem a localização em dados de verificação separados. Não há um limite de erro de aceitação definido neste manual: ele deve ser estabelecido conforme o objetivo de uso antes da avaliação. Caso o sinal seja não monotônico ou o ajuste não seja sustentado, manter os parâmetros iniciais como não validados e registrar a localização como estimativa de baixa confiança.
6. Após aprovação dos parâmetros, reiniciar o painel com as novas variáveis. Os testes de 1, 2, 3, 5 e 7 m em `dados/ensaios_rssi/RESULTADO_ENSAIOS_RSSI.txt` não sustentaram uma nova calibração confiável e possuem condições de montagem a confirmar. Não utilizar automaticamente os expoentes exploratórios desse relatório.

5.2 Operação diária

Barra lateral:

Quadro 17 – Controles da barra lateral e funções operacionais

Controle	Função
Enviar alerta de quedas	Liga/desliga o envio de notificações ao Telegram
Testar envio	Envia uma mensagem de teste ao Telegram
Limiar de alarme	Sensibilidade do detector. Menor = detecta mais e gera mais falsos alarmes; maior = o oposto (Seção 4.4)
Tempo para resetar alerta	Por quanto tempo o aviso de queda fica na tela (1–5 min)
Atualizar automaticamente / Intervalo	Atualização da tela (0,5–5 s)
Gravar amostras	Enquanto ativo, grava as amostras brutas (sensores e RSSI) em CSV e na tabela `amostra` do SQLite; os CSV são divididos em partes de 60 min
Reconectar	Refaz a conexão MQTT sem fechar o painel

Fonte: Elaborado pelos autores (2026).

O botão controla somente a gravação das amostras brutas. Sessões, janelas analisadas, possíveis quedas e status das antenas são registrados automaticamente no SQLite, mesmo quando a gravação está desligada.

Aba Monitoramento:

- Cartão do cinto: em situação normal, mostra o status normal. Em POSSÍVEL QUEDA, mostra em destaque a probabilidade, data, hora, antena mais próxima, quadrante e a contagem regressiva do aviso.
- Métricas: amostras recebidas, taxa (amostras/s), antenas online, Status Eletrodo e Status Telegram.

- Probabilidade de queda: gráfico das 8 últimas janelas (0–100%). Picos que cruzam o limiar geram alarme.
- Quedas Detectadas: tabela com cada evento e o status de envio ao Telegram.

Figura 9 – Barra lateral do painel



Fonte das figuras: elaborado pelos autores (2026).

1. Enviar alerta de quedas: liga ou desliga o envio de notificações ao Telegram.

2. Testar envio: envia uma mensagem de teste para verificar a configuração do Telegram.

3. Limiar de alarme: define a sensibilidade do detector de queda.

4. Tempo para resetar o alerta: controla por quanto tempo o aviso permanece na interface.

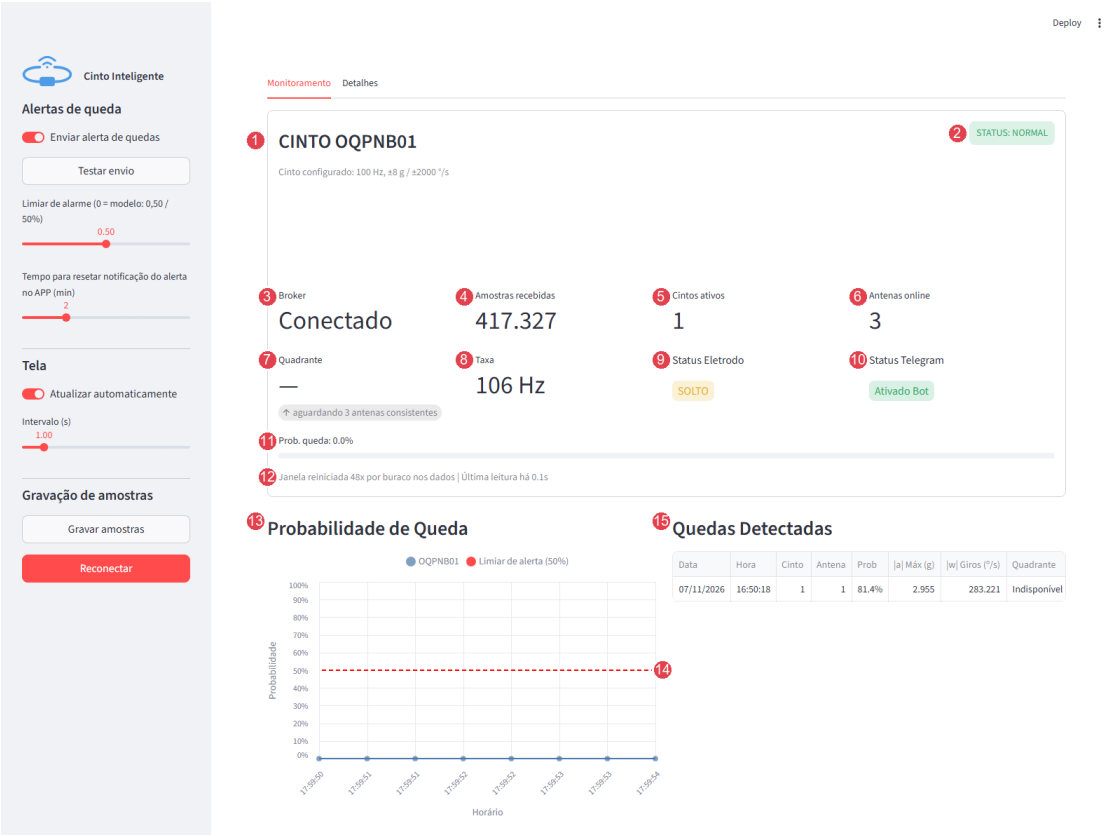
5. Atualizar automaticamente: liga ou desliga a atualização periódica do painel.

6. Intervalo (s): define o período entre as atualizações automáticas da tela.

7. Gravar amostras: inicia ou encerra a gravação das amostras brutas em CSV e na tabela `amostra` do SQLite.

8. Reconectar: refaz a conexão MQTT sem fechar o painel.

Figura 10 – Aba Monitoramento em operação



Fonte das figuras: elaborado pelos autores (2026).

1. Identificação do cinto: mostra o equipamento selecionado no painel.

2. Status do cinto: mostra STATUS NORMAL enquanto a probabilidade está abaixo do limiar. Quando o detector ultrapassa o limiar, o cartão muda para o alerta visual vermelho POSSÍVEL QUEDA e apresenta os dados do evento.

3. Broker: informa o estado da conexão MQTT com o computador central.

4. Amostras recebidas: apresenta o total de amostras recebidas pelo painel.

5. Cintos ativos: mostra quantos cintos estão enviando dados.

6. Antenas online: indica quantas antenas estão conectadas ao sistema.

7. Quadrante: indica a região estimada do ambiente: SE (superior esquerdo), SD (superior direito), IE (inferior esquerdo) ou ID (inferior direito). O traço nesta captura indica que a localização não estava disponível naquele instante.

8. Taxa: mostra a frequência atual de recebimento das amostras, em hertz.

9. Status do eletrodo: informa se os eletrodos estão CONECTADOS ou SOLTOS; nesta captura, aparecem soltos.

10. Status Telegram: informa se o bot está ativo para envio de notificações.

11. Probabilidade de queda: mostra a probabilidade atual e a barra visual correspondente.

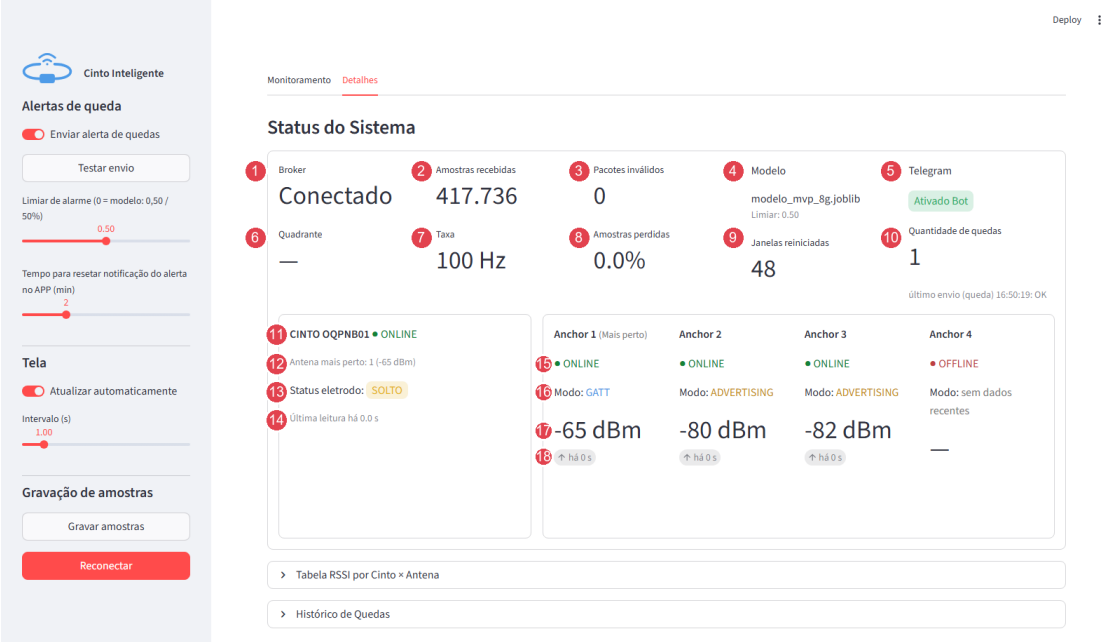
12. Diagnóstico da janela: informa quantas vezes a janela foi reiniciada por lacunas e há quanto tempo chegou a última leitura.

13. Gráfico de probabilidade: acompanha as últimas janelas analisadas.

14. Limiar de alerta: linha tracejada vermelha que marca o valor mínimo de probabilidade para gerar alerta. O controle Limiar de alarme na barra lateral ajusta esse valor; nesta captura, está em 50%.

15. Quedas detectadas: lista os eventos registrados e seus dados, como data, hora, cinto, antena, probabilidade e quadrante.

Figura 11 – Status do sistema, cinto e antenas



Fonte das figuras: elaborado pelos autores (2026).

1. Broker: indica o estado da conexão MQTT com o computador central.
2. Amostras recebidas: mostra o total de amostras recebidas pelo painel.
3. Pacotes inválidos: conta pacotes que não puderam ser interpretados.
4. Modelo: mostra o arquivo do modelo carregado e o limiar de detecção em uso.
5. Telegram: indica se o bot está ativo e mostra o resultado do último envio.
6. Quadrante: pode indicar SE (superior esquerdo), SD (superior direito), IE (inferior esquerdo) ou ID (inferior direito), conforme a posição estimada no cômodo. O traço nesta captura significa que não havia uma estimativa disponível naquele instante.
7. Taxa: mostra a frequência atual de recebimento das amostras, em hertz.
8. Amostras perdidas: indica a porcentagem de amostras não recebidas.
9. Janelas reiniciadas: conta reinicializações de janelas após interrupções no fluxo de dados.
10. Quantidade de quedas: mostra o total de possíveis quedas registradas.

11. Identificação e conexão do cinto: mostra o identificador do cinto e se está online.

12. Antena mais próxima: informa qual antena está recebendo o sinal mais forte e o RSSI correspondente.

13. Status do eletrodo: indica se os eletrodos estão conectados ou soltos.

14. Última leitura: mostra há quanto tempo o cinto enviou dados pela última vez.

15. Estado ONLINE: o indicador verde informa que a Anchor 1 está conectada.

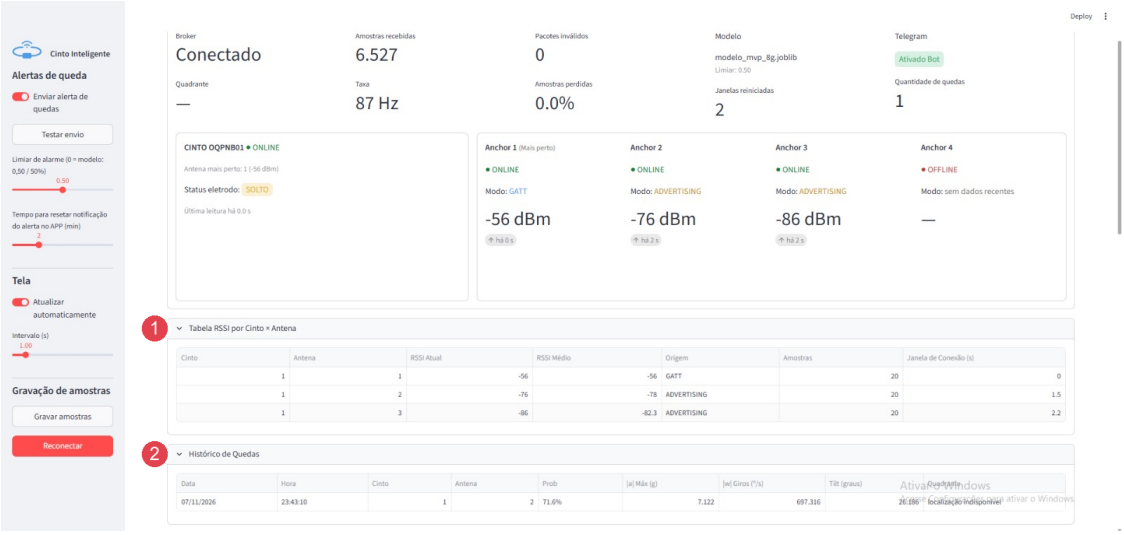
16. Modo: mostra como a antena recebe os dados. GATT usa uma conexão BLE para receber os dados completos dos sensores; Advertising recebe anúncios BLE sem conexão e permite medir o RSSI. Nesta captura, a Anchor 1 está em GATT.

17. RSSI (dBm): indica a intensidade recebida do sinal Bluetooth; valores menos negativos representam sinal mais forte.

18. Idade do RSSI: o selo cinza mostra há quantos segundos o valor do sinal foi atualizado.

As outras antenas exibem os mesmos campos e não foram numeradas individualmente.

Figura 12 – Tabelas RSSI e Histórico de Quedas

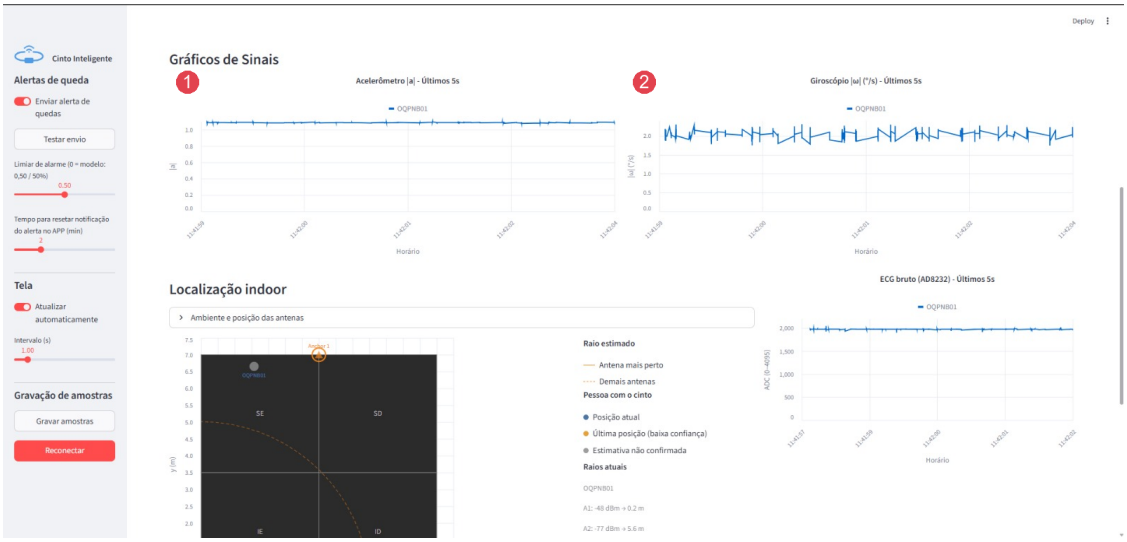


Fonte das figuras: elaborado pelos autores (2026).

1. Tabela RSSI por Cinto × Antena: apresenta, para cada antena, o RSSI atual e médio, a origem do dado (GATT ou Advertising), a quantidade de amostras e a janela de conexão.

2. Histórico de Quedas: lista os eventos registrados com data, hora, cinto, antena, probabilidade, aceleração máxima, giroscópio, inclinação e quadrante.

Figura 13 – Gráficos do acelerômetro e do giroscópio

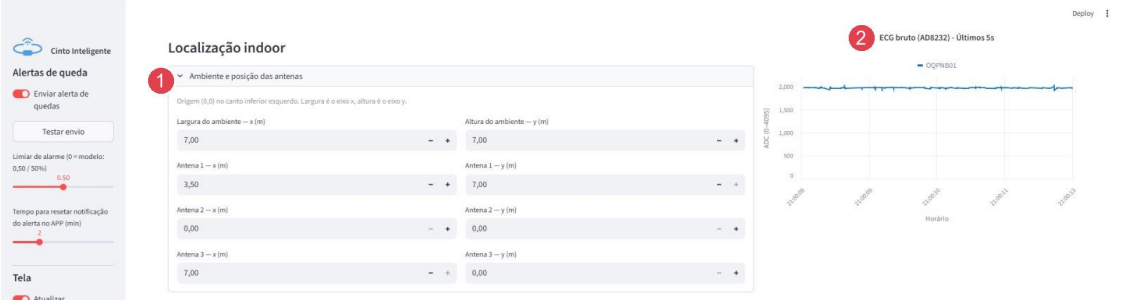


Fonte das figuras: elaborado pelos autores (2026).

1. Acelerômetro: mostra a magnitude da aceleração do cinto, em g, nos últimos cinco segundos.

2. Giroscópio: mostra a magnitude da velocidade angular, em graus por segundo, nos últimos cinco segundos.

Figura 14 – Configuração do ambiente e sinal de ECG

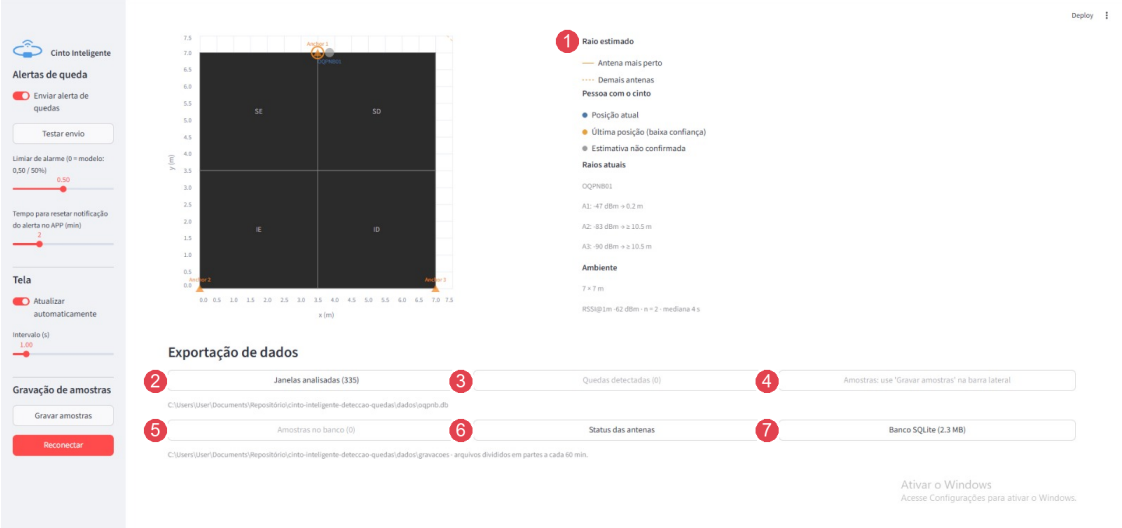


Fonte das figuras: elaborado pelos autores (2026).

1. Ambiente e posição das antenas: define as dimensões do cômodo e as coordenadas X/Y das três antenas, em metros.

2. ECG bruto: mostra os últimos cinco segundos do sinal captado pelo módulo AD8232.

Figura 15 – Mapa indoor e exportação de dados



Fonte das figuras: elaborado pelos autores (2026).

1. Raio estimado: cada círculo representa a distância estimada entre uma antena e o cinto, calculada a partir do RSSI. O círculo contínuo indica a antena mais próxima e os tracejados representam as demais. A estimativa é aproximada, pois o corpo e o ambiente alteram o sinal; por isso, interprete a posição como um quadrante, não como um ponto exato. As cores do marcador indicam: azul = posição confirmada; laranja = baixa confiança; cinza = estimativa bruta.

2. Janelas analisadas: baixa em CSV as janelas processadas na sessão.

3. Quedas detectadas: baixa em CSV os eventos de possível queda registrados.

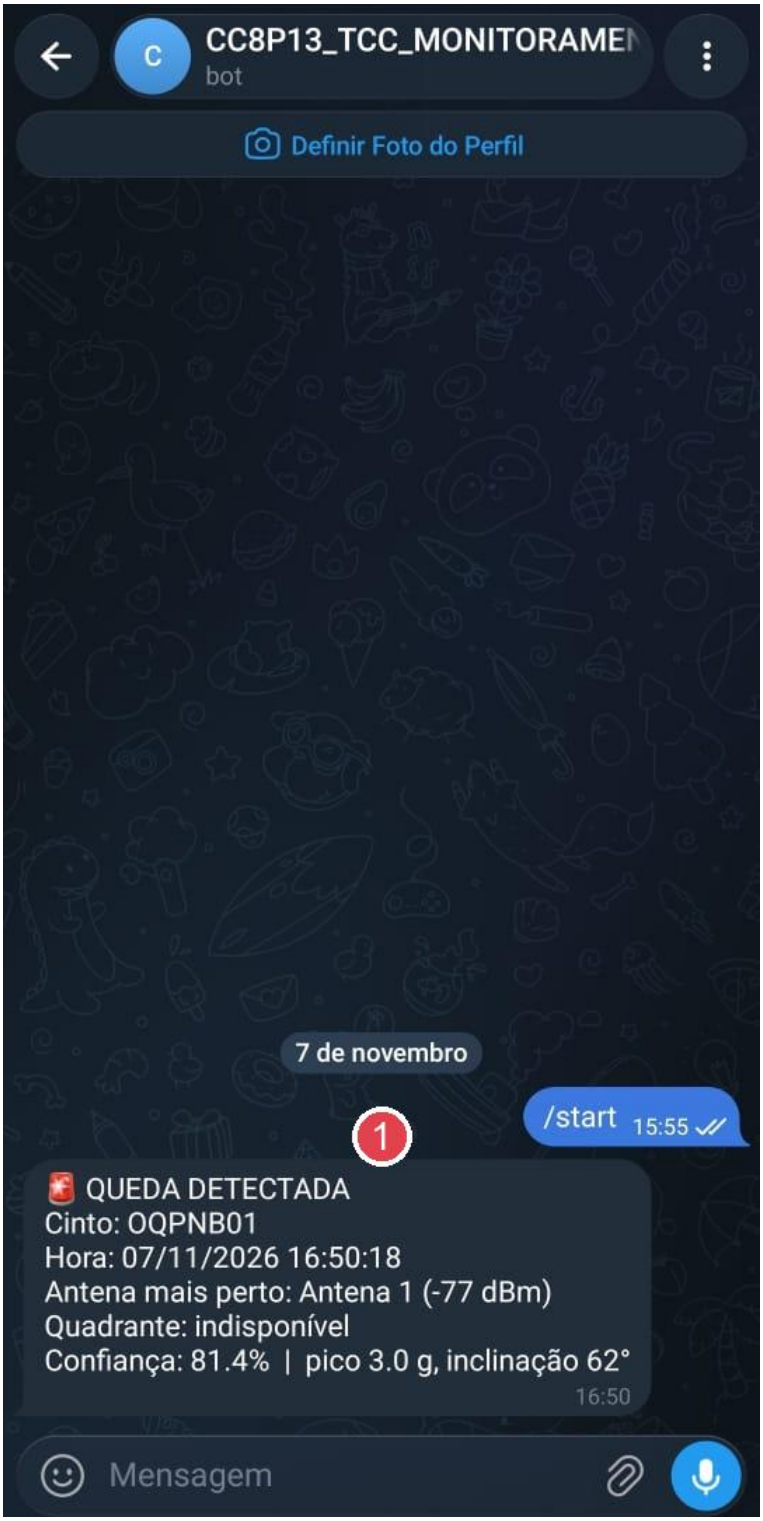
4. Amostras: antes de gravar, o botão orienta a iniciar Gravar amostras na barra lateral. Depois de gravar e parar, ele disponibiliza o CSV daquele período para download, dividido em partes de até 60 minutos. Esses arquivos podem ser usados em testes e reprocessamentos.

5. Amostras no banco: baixa em CSV as amostras gravadas no banco de dados.

6. Status das antenas: baixa em CSV o histórico de estado das antenas.

7. Banco SQLite: baixa o arquivo completo `.db` para consulta ou manutenção.

Figura 16 – Alerta de possível queda no Telegram



Fonte das figuras: elaborado pelos autores (2026).

1. Mensagem de alerta: apresenta o cinto, horário, antena mais próxima, quadrante, confiança e os dados de aceleração e inclinação associados ao evento.

5.2.1 Procedimento ao receber um alerta

Confira o cartão do cinto e o quadrante indicado e entre em contato com a pessoa ou vá até o local. O sistema é de apoio e não substitui o acompanhamento presencial.

5.3 Resolução de problemas

USB e rede Wi-Fi

Quadro 18 – Diagnóstico de USB e rede Wi-Fi

Categoria	Sintoma	Causa provável	Procedimento corretivo
USB / gravação	Placa não aparece em Portas (COM)	Driver ausente ou cabo só de carga	Instale o driver CP210x ou CH340 (Seção 3.2.2); troque por cabo de dados
USB / gravação	`Failed to connect to ESP32: Timed out`	Placa fora do modo de gravação	Segure BOOT durante `Connecting...`; no ESP32-C3, segure BOOT ao conectar o cabo
USB / gravação	Monitor Serial do cinto vazio	`USB CDC On Boot` desativado	Ative-o e grave de novo
USB / gravação	`Sketch too big`	Partição padrão pequena para Wi-Fi + BLE	Ferramentas → Partition Scheme → Huge APP (3MB No OTA)
Rede	Antena não conecta ao Wi-Fi	Rede de 5 GHz, SSID/senha errados ou sinal fraco	Use rede de 2,4 GHz; confira `WIFI_SSID`/`WIFI_PASSWORD`

Fonte: Elaborado pelos autores (2026).

MQTT, BLE e fluxo de dados

Quadro 19 – Diagnóstico de MQTT, BLE e fluxo de dados

Categoria	Sintoma	Causa provável	Procedimento corretivo
MQTT	Antena com Wi-Fi conectado, mas `[MQTT] Conectando ao broker ... Falhou (rc=...)`	IP do broker errado, firewall ou listener ausente	Confira `MQTT_BROKER` com `ipconfig`; regra de firewall da porta 1883; `listener 1883 0.0.0.0`; rede como Privada
MQTT	Painel mostra Erro MQTT	Mosquitto parado	`Restart-Service mosquitto` e clique em Reconectar
MQTT	Antena oscila entre online e offline	Alimentação instável ou Wi-Fi fraco	Carregue a bateria, confira os 5,0 V do MT3608, aproxime do roteador
Rede	“Antena online, mas nenhum pacote de cinto chegou ainda. Verifique no Serial da antena se ela mostra ‘GATT: CONECTADO’ e ‘TX MQTT’ > 0.” com o cinto ligado e conectado	Computador em rede Wi-Fi diferente da configurada nas antenas (o “online” pode ser a mensagem retida de uma sessão anterior)	Conecte o computador à mesma rede de `credenciais.h`; confira se `MQTT_BROKER` é o IPv4 atual do computador (`ipconfig`) e grave de novo se mudou
BLE	“Antena online, mas nenhum pacote de cinto chegou”	Cinto desligado, `CODIGO_AMBIENTE` diferente ou fora de alcance	Ligue o cinto; iguale o código nos dois firmwares; veja `[GATT] CONECTADO E AUTENTICADO!` no Serial da antena
Perda de pacotes	Taxa bem abaixo de 100 amostras/s e “amostras perdidas” crescendo	Interferência em 2,4 GHz, cinto longe da antena GATT, rede congestionada	Afaste as antenas de roteadores e micro-ondas; aproxime o cinto; reduza outros dispositivos na rede
Perda de pacotes	“Janelas reiniciadas” aumentando	Lacunas > 0,3 s no fluxo (troca de antena GATT)	Normal durante reconexões; se constante, siga o item anterior
Formato	“Pacotes com formato inválido”	Firmware de versão diferente do painel	Grave `cinto_mvp_v1` e `anchor_mvp_v1` deste repositório

Fonte: Elaborado pelos autores (2026).

Sensores, IA e localização

Quadro 20 – Diagnóstico de sensores, IA e localização

Categoria	Sintoma	Causa provável	Procedimento corretivo
Sensor	Status Eletrodo “Solto” / LED aceso	Eletrodo descolado, gel seco ou cabo P2 frouxo	Troque os eletrodos e encaixe o P2 até o fim
Sensor	ECG muito ruidoso ou saturado	Movimento, mau contato ou interferência de 60 Hz	Limpe a pele, use eletrodos novos, mantenha o usuário parado na verificação
Sensor	Aceleração em repouso longe de 1 g	MPU solto no case ou faixa alterada no firmware	Fixe o módulo; confirme ±8 g / ±2000 °/s em `iniciarMPU()`
IA	Muitos falsos alarmes	Limiar baixo ou atividades bruscas	Aumente o limiar (0,6–0,7); veja a tabela da Seção 4.4
IA	Quedas não detectadas	Limiar alto, cinto mal posicionado ou faixas do MPU diferentes das do modelo	Reduza o limiar; cinto na cintura, à frente; retreine se mudar as faixas
IA	“Modelo indisponível”	Arquivo `.joblib` ausente ou versão do scikit-learn diferente	Confira `modelos/modelo_mvp_8g.joblib`; reinstale com `pip install -r requirements.txt`
Localização	Quadrante instável ou errado	Sombreamento corporal, antenas colineares, calibração ausente	Reposicione as antenas fora de linha; calibre o RSSI (Seção 5.1); interprete como região

Fonte: Elaborado pelos autores (2026).

Telegram e operação

Quadro 21 – Diagnóstico de Telegram e operação

Categoria	Sintoma	Causa provável	Procedimento corretivo
Telegram	Toggle desabilitado	Variáveis `OQPNB_TG_TOKEN`/`OQPNB_TG_CHAT` não definidas	Defina-as e reabra o painel na mesma janela do PowerShell
Telegram	Erro 403 – bot was blocked by the user	O destinatário bloqueou o bot	No Telegram, abra o bot, desbloqueie e envie `/start`
Telegram	Erro 401 / 404	Token inválido ou revogado	Gere um novo token no @BotFather
Energia	ESP32 reinicia sozinho (brownout)	Bateria fraca ou MT3608 abaixo de 5 V	Carregue a bateria; reajuste o MT3608 para 5,0 V
Painel	`Port 8501 is already in use`	Outra instância do Streamlit aberta	Feche a outra janela ou use `--server.port 8502`

Fonte: Elaborado pelos autores (2026).

5.4 Desligamento seguro e manutenção preventiva

Desligamento:

1. Se houver gravação em andamento, clique em Parar gravação.
2. Desligue a chave gangorra do cinto e depois das antenas.
3. No PowerShell do painel, pressione Ctrl + C. O banco é gravado em lotes de no máximo 1 s, então os dados já estão salvos ao encerrar.
4. Retire os eletrodos do usuário e descarte-os (são de uso único).

Manutenção preventiva:

Quadro 22 – Rotinas de manutenção preventiva

Frequência	Ação
A cada uso	Eletrodos novos; limpar a pele com álcool 70% antes de aplicar
Diária	Recarregar cinto e antenas pelo USB-C do TP4056 (LED vermelho = carregando; azul/verde = carga completa)
Semanal	Limpar o case e o cabo P2 com pano levemente umedecido em álcool 70%, com o dispositivo desligado; nunca molhar
Semanal	Backup de `dados/oqpnb.db` e `dados/gravacoes/` com o painel fechado (ou pelo botão de exportação do `.db`)
Mensal	Abrir os cases e conferir soldas, fixação do MPU6050 e estado das baterias (sem estufamento); medir os 5,0 V do MT3608
A cada novo ambiente	Reconfigurar o ambiente e recalibrar o RSSI (Seção 5.1)
Ao atualizar o firmware	Gravar cinto e antenas com a mesma versão do repositório (Seção 3.2); manter o mesmo `CODIGO_AMBIENTE`
Ao atualizar o software	`git pull`; `pip install -r requirements.txt`; rodar os testes automatizados
Periodicamente	Trocar o token do Telegram se ele tiver sido exposto e nunca versioná-lo

Fonte: Elaborado pelos autores (2026).

5.5 Pendências de validação antes da entrega

- Arquitetura documental: alinhar a parte escrita à versão descrita na Seção 1.1, sem atribuir ao protótipo cálculo de BPM ou recursos não implementados.
- Montagem elétrica: confirmar a posição real da chave e corrigir a divergência entre Figuras 5/6 e tabelas; identificar os reguladores e validar proteção, recarga e isolamento entre USB e bateria.
- Evidências experimentais: registrar corrente média e de pico, autonomia, recarga e temperatura; medir latência ponta a ponta e erro de localização com posições de referência. As estimativas e verificações visuais não substituem esses ensaios.
- Comparação de modelos: executar ou documentar a comparação com limiares sob o mesmo protocolo de avaliação; não inferir superioridade sem resultados.
- Uso de IA: conferir ferramentas, versões, períodos, registros e assinatura; definir método para percentuais ou substituí-los, com aprovação da equipe, por descrição qualitativa verificável.
- Normalização e bibliografia: conferir as fontes primárias e a edição do manual de normalização exigida pela UNIP. As cinco seções seguem o apoio

de entrega, mas a conformidade integral com as normas vigentes não está certificada sem essa conferência.

5.6 Registros audiovisuais e rastreabilidade dos testes

Os vídeos em dados/testes podem complementar a documentação como evidências visuais da execução dos procedimentos, após conferência de seu conteúdo. Não substituem os dados brutos, a matriz de confusão, as medições elétricas ou os resultados de localização. A existência do arquivo e seu título, isoladamente, não comprovam que o detector reconheceu uma queda, que o alerta foi entregue ou que a posição estimada estava correta.

Quadro 23 – Registros audiovisuais disponíveis para conferência dos testes

Arquivo	Identificação pelo nome do arquivo	Correspondência com dados e resultados
Vídeo Andando perto da 3 Anchors.mp4	Atividade identificada como caminhada próxima às antenas	Conferir conteúdo e associar ao CSV e ao intervalo do ensaio; vínculo não confirmado neste manual
Vídeo Sentando perto das 3 Anchor.mp4	Atividade identificada como sentar próximo às antenas	Conferir conteúdo e associar ao CSV e ao intervalo do ensaio; vínculo não confirmado neste manual
Caindo.mp4	Registro identificado como queda	Conferir se a gravação mostra o movimento, o painel e o evento; vínculo com resultado do detector não confirmado
Deitando e Caindo.mp4	Registro identificado como deitar e cair	Conferir os intervalos de cada movimento e seus eventos correspondentes; vínculo com resultado do detector não confirmado

Fonte: Elaborado pelos autores (2026).

Para usar esses registros como comprovação de um teste específico, documentar o arquivo de vídeo, o intervalo no vídeo, a data e o horário do ensaio, o CSV correspondente, o identificador do cinto e das antenas, a posição de referência, o limiar utilizado e o resultado esperado/observado. Quando houver evento no banco ou no painel, registrar também sua identificação e verificar a sincronização dos horários. Não presumir correspondência com os ensaios RSSI apenas pela proximidade das antenas ou pelos nomes dos arquivos.

No TCC, citar o registro associado ao procedimento e ao resultado discutidos, distinguindo demonstração qualitativa de avaliação quantitativa. Se o vídeo não mostrar simultaneamente o movimento e a saída do sistema, complementar com dados sincronizados; não afirmar que ele comprova reconhecimento correto ou

tempo de resposta. Antes de publicar, conferir autorização das pessoas filmadas e remover dados identificáveis, credenciais ou informações sensíveis desnecessárias.

Os links apontam para os arquivos relativos ao manual no repositório. Para acesso local, conservar a estrutura de pastas; alguns leitores de PDF restringem a abertura de arquivos externos. No GitHub, a disponibilidade depende de os vídeos terem sido publicados e de as permissões de acesso permitirem a visualização pela banca.

REFERÊNCIAS

As referências abaixo correspondem às obras citadas neste manual e foram recuperadas da parte escrita de apoio. Os dados bibliográficos devem ser conferidos nas fontes primárias antes da entrega; não se presume essa verificação apenas por constarem no apoio.

BREIMAN, L. Random forests. **Machine Learning**, v. 45, n. 1, p. 5–32, 2001. DOI: 10.1023/A:1010933404324.

GIFFINGER, R.; FERTNER, C.; KRAMAR, H.; KALASEK, R.; PICHLER-MILANOVIĆ, N.; MEIJERS, E. **Smart cities: ranking of European medium-sized cities**. Viena: Centre of Regional Science, Vienna University of Technology, 2007.

GUBBI, J.; BUYYA, R.; MARUSIC, S.; PALANISWAMI, M. Internet of Things (IoT): a vision, architectural elements, and future directions. **Future Generation Computer Systems**, v. 29, n. 7, p. 1645–1660, 2013. DOI: 10.1016/j.future.2013.01.010.

PEDREGOSA, F. et al. Scikit-learn: machine learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011.

SUCERQUIA, A.; LÓPEZ, J. D.; VARGAS-BONILLA, J. F. SisFall: a fall and movement dataset. **Sensors**, v. 17, n. 1, art. 198, 2017. DOI: 10.3390/s17010198.

ZANELLA, A.; BUI, N.; CASTELLANI, A.; VANGELISTA, L.; ZORZI, M. Internet of Things for smart cities. **IEEE Internet of Things Journal**, v. 1, n. 1, p. 22–32, 2014. DOI: 10.1109/JIOT.2014.2306328.